LUDWIG-MAXIMILIANS-UNIVERSITÄT MÜNCHEN

CHAIR OF THEORETICAL COMPUTER SCIENCE AND THEOREM PROVING



Designing a Competitive Proof Verification System for TPTP-Style Benchmarks

Alexander Giller

Master's Thesis
Computer Science

Supervisor: Prof. Dr. Jasmin Blanchette

Advisor: Tanguy Bozec

Submission Date: November 2, 2026

Eidesstattliche Erklärung / Declaration of Authorship

Ich erkläre hiermit, dass ich die vorliegende Arbeit selbstständig und nur mit den angegebenen Quellen und Hilfsmitteln verfasst habe. Alle Passagen, die ich wörtlich oder sinngemäß aus der Literatur oder aus anderen Quellen übernommen habe, sind als solche mit Quellenangabe gekennzeichnet. Die Arbeit wurde in gleicher oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegt. Diese gedruckte Fassung ist ein Ausdruck der eingereichten elektronischen Fassung.

I hereby declare that I have written this thesis independently and only with the sources and aids specified. All passages that I have taken from literature or other sources, literally or in substance, have been marked as such with a reference to the source. This thesis has not been submitted to any other examination authority in the same or a similar form. This printed copy is a printout of the submitted electronic version.

München, November 2, 2026

Ort, Datum / Place, date

Alexander Giller

Hinweis zur Nutzung von KI-Werkzeugen
Bei der Erstellung dieser Arbeit wurden Werkzeuge der künstlichen Intelligenz (KI), darunter große Sprachmodelle (LLMs) wie Claude Opus 5.5, zur Unterstützung bei Übersetzung und sprachlicher Überarbeitung eingesetzt, insbesondere bei Grammatik, Rechtschreibung, Klarheit und Lesbarkeit sowie bei der Umstrukturierung und Kürzung des Textes. Darüber hinaus wurden KI-gestützte Werkzeuge für die Code-Vervollständigung, die Datenauswertung und die Erstellung von Abbildungen und Visualisierungen verwendet. Alle KI-generierten oder KI-unterstützten Ergebnisse wurden vom Verfasser geprüft und kritisch bewertet. Der Verfasser trägt die volle Verantwortung für die Richtigkeit, die Interpretation und die endgültige Darstellung der Inhalte dieser Arbeit.

Note on the Use of AI Tools
Artificial intelligence (AI) tools, including large language models (LLMs) such as Claude Opus 5.5, were used in the preparation of this thesis to support translation and linguistic editing, including improvements to grammar, spelling, clarity, and readability, as well as restructuring and condensing the text. In addition, AI-based tools were used for code completion, data analysis, and the generation of figures and visualizations. All AI-generated or AI-assisted outputs were reviewed and critically evaluated by the author. The author takes full responsibility for the accuracy, interpretation, and final presentation of the content of this thesis.

Acknowledgments

I would like to thank my advisor Tanguy Bozec for his guidance throughout this thesis, for the many helpful discussions and for his detailed and honest feedback, which shaped both the verifier and the way this thesis is written. I also thank Prof. Dr. Jasmin Blanchette for the opportunity to write this thesis at the Chair of Theoretical Computer Science and Theorem Proving.

Finally, I want to thank my wife, for proofreading the thesis and for supporting me with patience during the months of working on it.

Abstract

Automated theorem provers are very effective, but whether their results can be trusted depends on an independent check of the proofs they produce. This thesis presents the design and implementation of TSTPy, a Python-based proof verifier for TPTP/TSTP proofs, developed for ProoVer 2026, a proof-checking competition held as part of the FLoC Olympics 2026 in Lisbon (Cailler & Guilloud 2026c).

The scoring scheme of the competition weights a wrong acceptance ten times as heavily as a correct one. TSTPy therefore accepts a proof only if every step has been validated, rejects it only if it has found a concrete defect and returns *Unknown* otherwise. Structural, propositional, quantifier and clausal rules as well as Skolemization are checked internally, and every failing step gets its own diagnostic. The remaining steps are passed to the external prover Vampire, which only sees the premises of the step and never the proof as a whole.

Of the 91 problem/proof pairs of the competition, TSTPy accepts all 44 valid proofs, rejects 46 of the 47 invalid proofs and returns *Unknown* for the remaining one, which gives 136 of 138 points without a wrong acceptance. The median run time is 1.52 s per proof, well within the limit of 30 seconds. A generated suite of rule-level test proofs and 361 mutants of valid proofs, of which every mutant that is no longer valid is rejected, suggests that the checks are not simply tuned to the benchmark, and even without the external prover no invalid proof is accepted.

Keywords: Automated Theorem Proving, Proof Verification, TPTP, TSTP, Fail-Closed Checking, Skolemization, ProoVer 2026.

Contents

1 Introduction **1**
- 1.1 Motivation . . . 1
- 1.2 Research Questions and Scope . . . 1
- 1.3 Contributions and Structure . . . 1

2 Background **2**
- 2.1 Automated Theorem Proving and Proof Certification . . . 2
- 2.2 Proof Verifiers . . . 3
- 2.3 First-Order Logic and Skolemization . . . 3
- 2.4 The TPTP Formats . . . 4
- 2.5 The ProoVer 2026 Competition . . . 6

3 Related Work **6**
- 3.1 Approaches to Proof Checking . . . 7
- 3.2 Systems of ProoVer 2026 . . . 7
- 3.3 Positioning of This Thesis . . . 8

4 Task and Requirements **10**

5 Design **11**
- 5.1 Principles and Architecture . . . 11
- 5.2 Proof Model and Global Checks . . . 12
- 5.3 Step Dispatch and Rule Checkers . . . 12
- 5.4 Checking Skolemization . . . 14
- 5.5 Delegation, Verdicts and Soundness . . . 14

6 Implementation **15**
- 6.1 Overview and Parser . . . 15
- 6.2 Driver and Checkers . . . 16
- 6.3 Example of Skolemization in `PRV008` . . . 17
- 6.4 Resource Control and Tests . . . 17

7 Evaluation **18**
- 7.1 Setup . . . 18
- 7.2 Correctness and Score . . . 19
- 7.3 Rule Coverage and Modularity (RQ1, RQ2) . . . 19
- 7.4 Run Time and Memory (RQ3) . . . 21
- 7.5 Robustness (RQ3) . . . 22
- 7.6 Ablations . . . 24
- 7.7 Remaining Errors and Threats to Validity . . . 24

8 Discussion **25**
8.1 Interpretation of the Results 25
8.2 Trusted Computing Base and Design Trade-offs 25
8.3 Transfer to Other Provers: A Case Study with Vampire 26
8.4 Limitations 27

9 Conclusion and Future Work **28**

A Reproduction **29**

B Example Report **29**

C Results per Proof **30**

Bibliography **37**

1 Introduction

1.1 Motivation

Automated theorem proving (ATP) is used in many areas, from formal reasoning in mathematics to software verification and knowledge-based systems (Robinson & Voronkov 2001). The more complex ATP systems become, the more likely they are to contain bugs, and a short answer such as "the conjecture is a theorem" is then hard to trust (Taprogge et al. 2026a). A common solution is to let the prover output a derivation, also called a *proof object*, that justifies its answer step by step, and to check it with a separate and much simpler program, namely a *proof checker* or *proof verifier*. Trust then no longer rests on a large prover whose behavior becomes harder to follow with every optimization, but on a small checker that is easy to inspect. This idea goes back to the de Bruijn criterion and the LCF approach (Section 2.1).

In the TPTP ecosystem (Sutcliffe 2024), derivations are exchanged in the TSTP format (Sutcliffe 2010, 2007). A TSTP derivation is a directed acyclic graph of formulas in which each node records the rule that produced it and the premises it depends on. In the TPTP world, the established checker for such derivations is GDV (Sutcliffe 2006, 2026a). It asks trusted external ATP systems to confirm each step. This approach works for every inference rule, but it is slow and depends on the external systems. As a consequence, more recent checkers handle frequent rules internally (Chapter 3).

The first edition of the ProoVer competition turns this setting into a benchmark (Cailler & Guilloud 2026c). Each checker receives a first-order (FOF) problem and a TSTP proof, some of which contain deliberate errors, and has to classify the proof as valid or invalid, or abstain. Because a wrongly accepted proof costs ten times as much as a correctly accepted proof gains, a checker that cannot decide a step faces a genuine trade-off. Accepting pays off only if the proof is very likely valid, rejecting only if it is likely invalid, and in between abstaining is the better choice.

1.2 Research Questions and Scope

The objective of this thesis is to design and implement a Python-based verifier for TSTP derivations that is sound and competitive. *Sound* means that each accepted step is justified by the semantics of its rule, and *competitive* means a high score under the ProoVer 2026 scoring scheme (Chapter 4) within the wall-clock limit of 30 seconds per proof. The scope is limited to FOF problems and the inference rules of the ProoVer 2026 benchmark.

RQ1 Which inference rules must a verifier support to cover the ProoVer 2026 benchmark, and which can be left to an external prover?

RQ2 How can a modular architecture keep the trusted part of the verifier small and easy to extend?

RQ3 How do correctness, run time and memory consumption behave on the benchmark and on malformed derivations?

1.3 Contributions and Structure

The thesis contributes (i) a fail-closed, three-valued verdict policy derived from the scoring scheme of the competition, (ii) TSTPy, a modular verifier for TSTP derivations with internal checks for

most rules, a specific Skolemization check and a controlled fallback to the prover Vampire, and (iii) an evaluation on the ProoVer 2026 benchmark, on generated rule-level tests and on mutated proofs.

Chapter 2 introduces the logical background, the TPTP/TSTP formats and the competition, and Chapter 3 discusses related checkers. Chapter 4 states the task and the verdict policy, Chapters 5 and 6 present the design and its implementation. Finally Chapter 7 reports the experiments, whereas Chapter 8 discusses the arising implications and limits, while Chapter 9 concludes the thesis.

2 Background

This chapter introduces proof certification and the notion of a proof verifier, recalls the parts of first-order logic and Skolemization that the verifier depends on, describes the TPTP formats and explains the setting of ProoVer 2026.

2.1 Automated Theorem Proving and Proof Certification

For first-order logic the dominant automated provers are saturation provers based on resolution (Robinson 1965) and superposition, such as Vampire (Riazanov & Voronkov 2002) and E (Schulz et al. 2019). They combine many techniques such as clause selection heuristics, indexing and splitting, and are therefore very complex. Vampire in particular has dominated the yearly CASC competition for many years and won all eight divisions of CASC-30 in 2025 (Sutcliffe 2016, 2025). But trusting such a system blindly is difficult, because a single implementation error can lead to a wrong answer without any visible sign. The formal verification of proofs produced by the prover Leo-III, for example, uncovered several bugs in Leo-III itself (Taprogge et al. 2026a).

Proof verification separates *proof search* from *proof checking*. The prover produces a proof object, and an independent checker validates each inference step. The most important property of the checker is *soundness*, which means that it never accepts an invalid proof. Because the checker is much simpler than the prover, the trusted computing base (TCB) moves from the prover to a much smaller program (Pollack 1998; Chihani et al. 2013).

This principle goes back to two traditions of interactive theorem proving. Barendregt and Wiedijk named it the *de Bruijn criterion*: a system satisfies it if its proofs can be checked by a small and simple program that a sceptical user could inspect or even write himself (Barendregt & Wiedijk 2005). The *LCF approach* (Gordon et al. 1979) reaches the same goal by making theorems values of an abstract data type whose only constructors are the primitive rules of the logic. Every theorem is therefore built by a small trusted kernel. The kernel of HOL Light, for example, has only a few hundred lines of code (Harrison 2009).

For checking automated proofs, the analogy is not perfect. An LCF kernel knows a handful of primitive rules, whereas a TSTP derivation consists of coarse steps with many different rule names. A checker can implement many of these rules itself, which makes it larger than a kernel, or delegate them to an external prover, which brings a full proof search back into the trusted part. Balancing these two options is the central design question of this thesis.

2.2 Proof Verifiers

GDV calls itself a *derivation verifier* (Sutcliffe 2006), whereas ProoVer and Nörgler speak of *proof checkers* (Taprogge et al. 2026b). In this thesis both words are used as synonyms. A *proof verifier* takes a problem and a derivation and returns a verdict. The problem consists of a set of axioms and a conjecture, and the derivation claims to prove the conjecture from the axioms. In ProoVer the verdicts are `VerifiedGood`, which means that the derivation is a correct proof of the conjecture, `VerifiedBad`, where the derivation is an incorrect proof, and the abstentions `Unknown` and `Timeout`. A derivation is *valid* for a problem if

1. its names are unique, every parent exists and its dependency graph is acyclic (*structure*),
2. every imported axiom and conjecture is a copy of a formula of the problem (*faithfulness*),
3. every step is correct with respect to its declared status (*logical correctness*, Section 2.4), and
4. it is a refutation that negates the conjecture and derives `$false`.

These groups correspond to the taxonomy of checks of Taprogge et al. (2026b) (Table 4).

A verifier is *sound* if it returns `VerifiedGood` only for valid derivations, and *complete* for a class of derivations if it returns `VerifiedGood` for every valid derivation of that class. The GDV documentation uses the two words from the point of view of faults. Its "complete" (finding every fault) therefore corresponds to sound here (Sutcliffe 2026a). Whether a step follows from its parents is only semi-decidable in first-order logic. A verifier that asks a prover can therefore not always answer within a time limit. It therefore needs the abstentions and a policy for using them. This thesis uses a *fail-closed* policy, under which a step or proof that cannot be established is never reported as `VerifiedGood`.

2.3 First-Order Logic and Skolemization

Only the notions needed in this thesis are recalled here. Refer to Fitting (1996) for a full treatment. Terms are built from variables and function symbols, atoms from predicate symbols applied to terms or from equations, and formulas from atoms, $\top$, $\bot$, the connectives $\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$ and the quantifiers $\forall$ and $\exists$. Two formulas that differ only in the names of bound variables are *alpha-equivalent*, which is the notion of equality the verifier uses to compare formulas. A set of sentences Γ entails φ if every model of Γ satisfies φ, and

$$\Gamma \models \varphi \quad \text{iff} \quad \Gamma \cup \{\neg\varphi\} \text{ is unsatisfiable.} \tag{1}$$

Most provers use this *refutation principle* (Robinson 1965). They negate the conjecture, add it to the axioms and derive a contradiction, which is why TSTP proofs end in $\bot$ (Sutcliffe et al. 2006).

The *polarity* of a subformula is positive if it lies under an even number of negations, counting the antecedent of an implication as a negation, and negative otherwise (Nonnengart & Weidenbach 2001). A quantifier at a negative position behaves like its dual. In $(\exists x\, p(x)) \Rightarrow q$, for example, the existential quantifier is effectively universal. Below an equivalence a subformula occurs in both polarities.

Normal forms and Skolemization. Many provers transform their input into clause normal form by eliminating implications and equivalences, pushing negations inward, eliminating existential quantifiers and distributing disjunctions. To avoid an exponential blow-up, they replace subformulas by fresh predicates with a defining equivalence (Tseitin 1983; Plaisted & Greenbaum 1986; Nonnengart & Weidenbach 2001). Such definitions introduce new symbols, and the result is only equisatisfiable with the input.

Skolemization eliminates an existential quantifier. An existential variable y in the scope of the universal variables $x_1, \ldots, x_n$ is replaced by a term $f(x_1, \ldots, x_n)$ with a fresh *Skolem function* f:

$$\forall x_1 \ldots \forall x_n \, \exists y \, \varphi(x_1, \ldots, x_n, y) \;\rightsquigarrow\; \forall x_1 \ldots \forall x_n \, \varphi(x_1, \ldots, x_n, f(x_1, \ldots, x_n)). \tag{2}$$

The result is not equivalent to the original formula, but equisatisfiable, which is enough for a refutation. For checking this has a central consequence. The Skolemized formula entails the original one, but a Skolem term with too few arguments yields an even stronger formula that still entails the original one and is no longer equisatisfiable. A Skolemization step can therefore not be verified by an entailment test. An external tool could still check it by testing equisatisfiability instead of entailment, but this is a harder question than entailment, and the ProoVer rules require that Skolemization is checked internally. An internal checker has to validate the shape of the transformation, the dependencies of the Skolem term on the universal variables in scope and the *freshness* of the symbol (Nonnengart & Weidenbach 2001).

For example, the correct Skolemization of $\forall x \exists y \, parent(y,x)$ is $\forall x \, parent(f(x),x)$. The formula $\forall x \, parent(c,x)$ also entails the axiom, but together with $\forall x \neg parent(x,x)$ it yields the contradiction $parent(c,c)$, although the original formulas are satisfiable. Freshness matters for the same reason. Replacing y in $\exists y \, p(y)$ by a constant a that already occurs in an axiom $\neg p(a)$ yields a contradiction from satisfiable premises.

2.4 The TPTP Formats

The TPTP library is the standard collection of problems for evaluating ATP systems under comparable conditions (Sutcliffe 2017). It comes with the TSTP library of solutions (Sutcliffe 2026e), the SZS ontologies for results and online services such as SystemOnTPTP (Sutcliffe 2000). The language has dialects for untyped first-order formulas (`fof`), clauses (`cnf`) and typed and higher-order logic. ProoVer 2026 and this thesis use `fof`.

The FOF language. A TPTP file is a sequence of *annotated formulas* of the form `fof(Name, Role, Formula, Source, UsefulInfo)` (Sutcliffe 2026c). The roles that occur in the benchmark are listed in Table 2. Variables start with an upper-case letter, all other symbols with a lower-case letter, and Table 1 lists the logical symbols. A quantified body that is a binary formula needs parentheses, and the associative connectives `&` and `|` must not be mixed without them.

Table 1: Logical symbols of the FOF language (Sutcliffe 2026c).

Notion	Symbol	TPTP	Example
negation	$\neg$	`~`	`~p(X)`
conjunction, disjunction	$\wedge$, $\vee$	`&`, `|`	`p(a) & q`
implication, reverse implication	$\Rightarrow$, $\Leftarrow$	`=>`, `<=`	`p => q`
equivalence, exclusive or	$\Leftrightarrow$, $\oplus$	`<=>`, `<~>`	`p <=> q`
NOR, NAND	$\downarrow$, $\uparrow$	`~|`, `~&`	`p ~| q`
universal, existential quantifier	$\forall$, $\exists$	`!`, `?`	`![X]: p(X)`
equality, inequality	$=$, $\neq$	`=`, `!=`	`f(X) != c`
truth constants	$\top$, $\bot$	`$true`, `$false`	`$false`

TSTP derivations. A derivation is written in the same language and uses the source field to record where each formula comes from (Sutcliffe et al. 2006; Sutcliffe 2026d). A *leaf* is either taken from the problem, `file(FileName, FormulaName)`, or introduced by the prover, `introduced(Type, ...)`. A derived formula carries an inference record `inference(Rule, UsefulInfo, Parents)`, whose rule name is not standardized. Its `UsefulInfo` holds the status of the step and, for Skolemization, the annotations `new_symbols(skolem, [sK0])` and `skolemize(Var, Term)`. The status is a value of the SZS ontology (Sutcliffe 2008, 2026b), and Table 2 summarizes the roles and statuses that occur in the benchmark.

Table 2: Roles and statuses in the ProoVer 2026 proofs (Sutcliffe 2026c,d).

Role or status	Meaning
`axiom`, `hypothesis`	An assumption of the problem, imported with a `file` source.
`conjecture`	The statement to be proved, imported from the problem.
`negated_conjecture`	The negation of the conjecture, from which the refutation starts.
`plain`	A derived formula without a special role.
`thm` (Theorem)	The formula is derived from its parents. Used by most rules.
`cth` (CounterTheorem)	The negation of the formula is derived from its parent, which must be the conjecture.
`esa` (EquiSatisfiable)	The formula is equisatisfiable with its single parent. Used for Skolemization.

By convention, each Skolemization step eliminates one variable, and every introduced symbol is globally new (Sutcliffe 2026d). These conventions make Skolemization checkable at all (Taprogge et al. 2026b), but provers do not always follow them (Section 8.3). Solvers report their result and delimit the proof with SZS lines such as `% SZS status Theorem`, and ProoVer reuses this style for its verdicts. Figure 1 shows the smallest proof of the benchmark.

Soundness of refutations. Let Γ be the axioms and the negated conjecture of a derivation that ends in $\bot$. A step with status `thm` is sound if its conclusion is entailed by its premises. A step with status `esa` is sound if every model of its premises can be extended, by interpreting the fresh

```
fof(a1,axiom, ?[X]: ![Y]: ?[Z]: ![W]: r(X,Y,Z,W),
    file('Problems/PRV000+1.p',a1) ).
fof(c,conjecture, ?[X]: ![Y]: ?[Z]: ![W]: r(X,Y,Z,W),
    file('Problems/PRV000+1.p',c) ).
fof(neg,negated_conjecture,
    ![A]: ?[B]: ![C]: ?[D]: ~r(A,B,C,D),
    inference(negated_conjecture,[status(cth)],[c]) ).
fof(bot,plain, $false,
    inference(consequence,[status(thm)],[neg,a1]) ).
```

Figure 1: The ProoVer 2026 proof `PRV000+1.s` (comments and SZS delimiters omitted). Axiom and conjecture are imported from the problem file, `neg` negates the conjecture and one `consequence` step derives `$false`.

symbols, to a model of the premises and the conclusion. If all steps are sound and the dependency graph is acyclic, an induction over the graph shows that Γ together with all derived formulas is satisfiable whenever Γ is. Since $\bot$ is derived, Γ is unsatisfiable, and by Equation 1 the conjecture follows from the axioms. The argument also shows that the check for a final `$false` is part of soundness, because a derivation of sound steps that never reaches $\bot$ proves nothing.

2.5 The ProoVer 2026 Competition

ProoVer is a competition for proof checkers in the TPTP ecosystem, inspired by CASC (Cailler & Guilloud 2026c). Participants receive correct and deliberately flawed proofs and must decide for each whether it is valid. The rules of the 2026 edition fix the following points.

- All proofs are refutations that end in `$false`.
- The import of axioms and conjectures (alpha-equivalent to the referenced formula), `negated_conjecture` and `skolemize` must be checked *internally*.
- All other rules may be checked by an external prover, but only for steps with status `thm` or `cth`, only with the premises of the step and never by re-proving the whole conjecture.

The benchmark was announced with 100 problems, half of them with a valid proof, and a wall-clock limit of 30 s per problem. Nine of the 100 pairs are not part of the published results. The scores therefore refer to the remaining 91 pairs (Cailler & Guilloud 2026b). A correctly rejected bad proof earns $+2$, a correctly accepted good proof $+1$, an abstention 0, a rejected good proof -1 and an accepted bad proof -10 (Cailler & Guilloud 2026a). The proofs use a natural-deduction-like vocabulary with rules such as `instantiate`, `modus_ponens`, `split_conjunct`, `skolemize` and the generic `consequence`, and some instantiation steps carry `new_symbols(herbrand, ...)` annotations.

3 Related Work

This chapter groups the approaches to checking proofs of automated provers, surveys the systems of ProoVer 2026 (Cailler & Guilloud 2026a), discusses GDV (Sutcliffe 2006) and Nörgler (Taprogge

et al. 2026b) in detail and positions this thesis.

3.1 Approaches to Proof Checking

Checkers for TSTP derivations differ mainly in how they decide that a single step is correct. *Semantic checkers* verify the structure of the derivation and ask a trusted ATP whether each conclusion follows from its premises, as GDV (Sutcliffe 2006) and Nörgler (Taprogge et al. 2026b) do. This works for any rule, but makes the ATP part of what has to be trusted. *Replay checkers* implement the rules themselves and recompute each conclusion, which is the standard for propositional proofs in DRAT and LRAT (Wetzler et al. 2014; Cruz-Filipe et al. 2017). *Certification* translates the derivation into a term for the small kernel of a logical framework such as Dedukti (Assaf et al. 2023; Delahaye et al. 2013; Sutcliffe et al. 2025), which gives the strongest guarantee at the cost of translation effort. Practical systems combine these strategies. Outside TPTP, the Alethe format and the checker Carcara play this role for SMT (Schurr et al. 2021; Andreotti et al. 2023), and Sledgehammer replays the output of external provers in the kernel of Isabelle (Böhme & Nipkow 2010).

Provers as step validators. Vampire (Riazanov & Voronkov 2002; Kovács & Voronkov 2013) plays two roles in this thesis. As a *step validator* it decides whether a single step follows from its premises, as in GDV and in most ProoVer entrants, and VaLeaDate additionally checks Vampire's output in Lean (Bodingbauer et al. 2026). A positive answer is evidence for a valid step, but a timeout says nothing. A verifier therefore cannot treat a missing answer as a rejection, and every call costs time from the limit of the proof. As a *proof producer* it is the subject of the transfer experiment in Section 8.3.

3.2 Systems of ProoVer 2026

ProoVer 2026 was held on 27 July 2026 in Lisbon, alongside CASC-J13 at FLoC 2026. Ten systems took part, eight in the regular division and GDV and GDV-LP in a demonstration division (Table 3). Most entrants delegate entailment steps to E or Vampire, and Nörgler and ProofCheck also use the model finder Mace4 to reject wrong steps. Only CheckProof and GAPT rely on internal reasoning engines. Several systems, among them ProofGuard and PyCheck, check Skolemization structurally instead of by entailment. These two are also the only other entrants written in Python.

GDV and Nörgler. GDV, developed by Geoff Sutcliffe, is the established verifier of the TPTP world (Sutcliffe 2006, 2026a). Its structural checks test that parents exist, that the derivation is acyclic and that refutations end in `$false`. For every formula it then creates a verification obligation that a trusted ATP discharges. A derived formula must follow from its parents, and a negated conjecture must be the negation of its parent. For a Skolemized formula the obligation is reversed, and the parent is proved from the conclusion, which the documentation itself calls incomplete (Sutcliffe 2026a). Without the Skolem annotations, GDV falls back to entailment in both directions, and the failure of this fallback does not reject the proof (Taprogge et al. 2026b). To reduce the trust in the ATP, every system serves as the trusted system for the derivations of the others, and GDV-LP replaces the trust by LambdaPi certificates (Sutcliffe 2026a; Sutcliffe et al. 2025).

Table 3: Systems entered in ProoVer 2026, based on their system descriptions (Cailler & Guilloud 2026a).

System	Language	Core approach	External tools
CheckProof 0.1	C	Own saturation engine	none
GAPT 2.20	Scala	LK proof, internal prover	none
GDV 2.0	C	Semantic verification	ATPs via SystemOnTPTP
GDV-LP 2.0	C, OCaml	GDV with LambdaPi certificates	Zenon Modulo, lambdapi
mrs-proover 0.2.0	Rust	Structural checks, ATP ladder	E, Vampire
Nörgler 1.1	Scala	Semantic verification	E, Vampire, Mace4
ProofCheck 1.0	C++	Internal clausal engine	E, Vampire, Mace4
ProofGuard 1.0	Python	Internal structural checks	E
PyCheck 0.1	Python	Based on PyRes	E
VaLeaDate 0.1	Scala, Lean	Vampire per step, Lean	Vampire, Lean

Nörgler (Taprogge et al. 2026b) builds on GDV and organizes the checks into global structure, local structure, logical correctness and faithfulness (Table 4). It checks Skolemization internally, uses E and Vampire for entailment and Mace4 to reject wrong steps, and adds checks that GDV lacked at the time. Of 1976 correct and 1952 falsified FOF proofs from the TSTP library, GDV and all but one configuration of Nörgler verify 1972 of the correct proofs, GDV rejects 1845 of the falsified proofs, and Nörgler between 1845 and 1952, at 0.08 to 0.31 s per correct proof against 0.54 s for GDV (Taprogge et al. 2026b). A direct comparison on the same benchmark was not possible, because these proofs use a different rule vocabulary than the ProoVer proofs.

Table 4 compares the checks of GDV, Nörgler and TSTPy. The checks NUF, DCP, APS and PIS make a proof *informative* rather than correct, and Nörgler omits them as well (Taprogge et al. 2026b). The comparison was useful during development, because it showed that an earlier version of TSTPy lacked the checks EIF and NAU.

3.3 Positioning of This Thesis

This thesis explores one particular trade-off. It is a modular Python checker in which every rule check is a small function that can be tested on its own, internal checks are tried before any prover is called and the verdict policy is derived explicitly from the scoring scheme. Semantic checkers keep their own code small, but they rely on an ATP, a far larger program, even for simple steps such as instantiating a universal formula. The internal checks of TSTPy take the place of the ATP for such steps. The code that decides most steps is therefore the smaller one. ProofGuard checks only the import steps, the negated conjecture and Skolemization internally and sends all other theorem steps to E, whereas TSTPy checks 20 rules internally and needs the prover for less than one percent of

Table 4: Checks of GDV, Nörgler and TSTPy, using the abbreviations of Taprogge et al. (2026b). "n.s.": not stated in the sources.

Check	Meaning	GDV	Nörgler	TSTPy
EIF	the proof ends in `$false`	yes	yes	yes
IAA	parents are acyclic	yes	yes	yes
NAU	names are unique	n.s.	yes	yes
NUF	no unused formulas	yes	no	no
IPE	parents exist	yes	yes	yes
ONU	only the negated conjecture uses the conjecture	later	yes	yes
DCP	each formula is `$false` or a parent	yes	no	no
ICN	conjecture correctly negated	yes	yes	yes
ICE	step correctly entailed	ATP	ATP	internal, ATP
ICS	step correctly Skolemized	inverse	yes	internal
APS, PIS	axioms and premises satisfiable	optional	no	no
COO	imported formula matches the problem	copy	equal	alpha-equiv.

the derived steps of the benchmark (Chapter 7).

4 Task and Requirements

This chapter states the task of TSTPy, derives the verdict policy from the scoring scheme and summarizes the requirements that the design has to meet.

Task and assumptions. The verifier receives a FOF problem file and a TSTP proof file and decides whether the proof is a valid refutation of the negated conjecture in the sense of Section 2.2. The proofs use the roles `axiom`, `conjecture`, `negated_conjecture` and `plain`, and the statuses `thm`, `esa` and `cth`. The rules fix the essential requirements and deliberately leave the handling of edge cases to the participants as part of the task. It is assumed that the problem file is trusted and the proof file is not, that proofs can be large and deeply nested and that an external ATP may or may not be available. Where the rules leave room for interpretation, the verifier follows the conservative reading and never accepts without evidence.

Scoring and the verdict policy. Under the ProoVer 2026 scheme a wrong acceptance costs -10, whereas a correct acceptance only gains $+1$, a correct rejection $+2$, a wrong rejection -1 and an abstention 0 (Section 2.5). A single wrong acceptance therefore cancels ten correct ones, while an abstention costs at most the points of one proof. The verifier has no way to estimate how likely an undecided step is to be valid. It therefore does not guess. It accepts a proof only if every step has been validated, rejects it only if it has found a concrete defect and abstains in all other cases, for example if no checker exists for a rule or the prover gives no answer. This *fail-closed*, three-valued policy is the central decision of the thesis. All other requirements follow from it or from the competition setting.

Requirements.

Soundness. A proof is accepted only if every step has been validated. Missing tools, tool errors and unparsable output never lead to acceptance. A step with an unknown rule name is accepted only if the external prover confirms that it follows from its premises.

Coverage. The verifier parses FOF problems and TSTP proofs with all annotations, checks the import of axioms and conjectures up to alpha-equivalence, the negated conjecture, the dependency graph, the final `$false`, and Skolemization internally, and checks the other rules of the benchmark internally where this is feasible.

Restricted delegation. Steps without an internal checker are passed to an external ATP only if their status is `thm` or `cth`, and only together with their own premises.

Resources. Every proof is processed within the wall-clock limit of 30 s, and malformed or deeply nested input does not crash the verifier.

Diagnostics and extensibility. Every finding names the step and a reason, and a rule can be added or tested without changing the checkers of the other rules.

5 Design

This chapter presents the architecture of TSTPy independently of its implementation, which follows in Chapter 6.

5.1 Principles and Architecture

Four principles follow from the requirements of Chapter 4.

Fail closed. Acceptance is the only outcome that needs positive evidence. Errors and missing tools lead to rejection or abstention, never to acceptance. A step with an unknown rule is accepted only if the prover confirms it.

Rule locality. Each rule is checked by a separate function that only sees the step and its parents and returns a list of issues.

Cheap checks first. Syntactic checks run before any external tool, and the ATP is only a fallback for steps that cannot be validated syntactically.

Diagnostics as data. Checkers return structured issues instead of printing them. Reports, summaries and tests thus use the same information. Appendix B shows an example report.

The benchmark uses more than thirty rule names without a reduction to primitive rules. An LCF-style kernel is therefore not an option. Instead, all rule checkers rely on the same few operations, namely alpha-equivalence, substitution, template matching and unification. These act as a kernel in a weak sense, because a defect in them affects every rule, whereas a defect in one rule checker stays local.

Architecture. Figure 2 shows the pipeline. A proof is checked in a separate worker process with a wall-clock limit. The parser turns the problem and the proof into a proof model, global checks validate the proof as a whole and a dispatcher passes each step to the checker of its rule or, if there is none, to the ATP. The verdict aggregation finally combines all findings into one verdict.

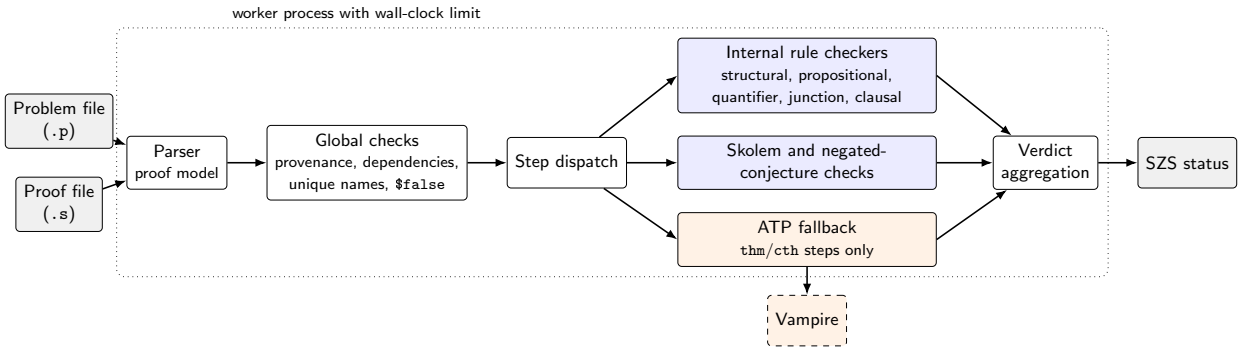


Figure 2: Architecture of TSTPy. Parsing, checking and verdict aggregation run in a worker process with a wall-clock limit. The internal rule checkers are grouped into families (Table 5).

5.2 Proof Model and Global Checks

An *annotated formula* has a name, a role, a formula and a justification, which is either a source in the problem file or an *inference record* with the rule name, the parent names, the status and the symbol annotations. Formulas are immutable syntax trees in which conjunctions and disjunctions are n-ary, which makes flattening explicit and simplifies the junction and clausal checks. Roles, statuses and rule names are enumerations. Unknown values are thus detected instead of silently ignored.

Four checks concern the proof as a whole. *Provenance* compares every formula imported with a `file(...)` source with the cited problem formula up to alpha-equivalence and compares the roles. The *dependency check* rejects cycles and any use of the conjecture except by its negation, because otherwise the proof would assume what it has to refute. *Unique names* rejects a proof in which a name is defined twice, and *termination* requires a derived step with the formula `$false`. Termination does not require that this step depends on the negated conjecture. A problem with contradictory axioms is therefore not rejected.

5.3 Step Dispatch and Rule Checkers

All step checkers have the same interface. They take a step and its parents and return a list of issues, each with the step name and a reason. Before calling a checker, the dispatcher checks what is common to all rules, namely that every parent exists, that the number of parents fits the rule and that the status is the one the rule requires. A checker never throws an exception because a proof is invalid, and an exception therefore indicates an input the verifier cannot process. The design foresees a registry that maps rule names to checkers. A new rule would then need only one entry

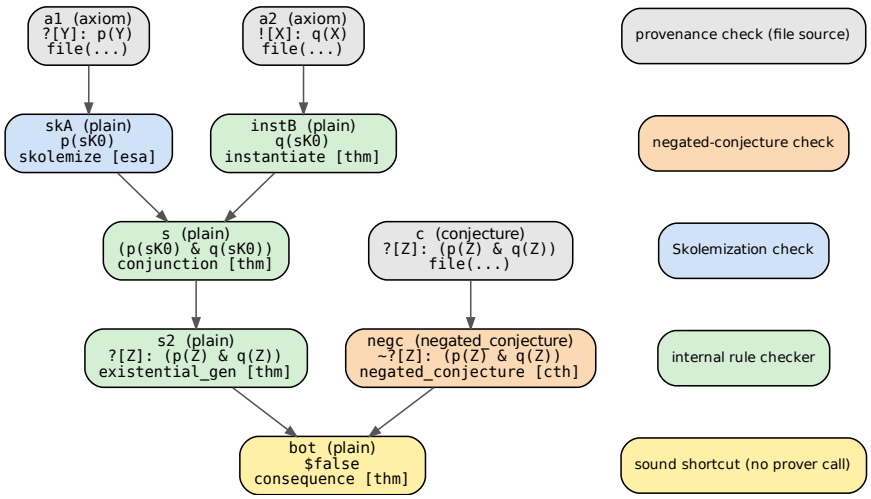


Figure 3: Dependency graph of the valid benchmark proof `PRV089`. Each edge leads from a parent to the step derived from it, and the color shows which check validates the node.

and one function.

Figure 3 shows a complete proof as the verifier sees it. `PRV089` proves the conjecture $\exists Z\,(p(Z) \wedge q(Z))$, that is, it refutes its negation, from the axioms $\exists Y\,p(Y)$ and $\forall X\,q(X)$. The step `skA` Skolemizes `a1` with a fresh constant *sK0*, `instB` instantiates `a2` with this constant, `s` combines the two facts and `s2` generalizes *sK0* back to an existential variable, which yields the conjecture itself. The final step `bot` has the conjecture and its negation as parents. Since they are complementary, it is decided without a prover, and the whole proof is verified by internal checks alone. Each step is judged only by its parents. The graph and not the order in the file therefore determines what has to hold.

Rule checkers. Table 5 groups the supported rules into families. Most families reduce to alpha-equivalence and structural matching. The checks are deliberately strict. They accept only the shape that the rule prescribes. A conclusion that follows from the parents, but not by the cited rule, is therefore rejected.

Table 5: Rule families and the principle of their checks.

Family	Rules	Principle
Structural	`copy`, `duplicate`, `rename_variable`, `double_negation`, `remove_double_negation`	Conclusion alpha-equivalent to the parent, possibly with one double negation added or removed
Propositional	`modus_ponens`, `conjunction`, `split_conjunct`	Conclusion is the consequent of an implication, the conjunction of the parents, or one of their conjuncts
Quantifier	`instantiate`, `existential_gen`	Conclusion is a substitution instance (or generalization) of the parent, found by template matching
Junction	`weaken`, `commute`, `excluded_middle`	Shape check after flattening, ATP fallback for unusual variants
Clausal, equational	`resolution`, `paramodulation`, `reflexivity`, `transitivity`, `rewrite`	Unification and construction of the expected resolvent or rewritten formula
Skolemization	`skolemize`	Section 5.4
Delegated	`consequence`, `add_hypothesis`, other rules with status `thm` or `cth`	Entailment from the premises by the ATP (Section 5.5)

The value of the rule checks shows in the invalid proofs of the benchmark. In `PRV056`, a step instantiates

$$\forall X\,\forall Z\; rel16(X,Z,sK0(X,Z)) \quad \text{to} \quad rel16(m_0,n_0,sK0(n_0,m_0)).$$

Matching binds $X \mapsto m_0$ and $Z \mapsto n_0$. The third argument would therefore have to be $sK0(m_0,n_0)$, and the step is rejected. The final `consequence` step of this proof would pass the prover, because the wrong instance still contradicts the negated conjecture. Only the rule check of the intermediate step reveals the error.

5.4 Checking Skolemization

Skolemization cannot be validated by an entailment test (Section 2.3), and the competition requires that it is checked internally. The checker therefore validates the following conditions, which together make the step satisfiability-preserving.

1. The step has exactly one parent, the status `esa` and declares exactly one new symbol of kind `skolem`.
2. The Skolem term consists of the declared symbol applied to exactly the universal variables in whose scope the eliminated quantifier occurs.
3. Replacing the quantifier and its variable by the Skolem term in the parent yields the conclusion up to alpha-equivalence.
4. The symbol is fresh. It neither occurs in the problem nor was it introduced by an earlier step.

The checker tracks the polarity while it traverses the parent. An existential quantifier at a negative position is effectively universal and must not be Skolemized, and a quantifier below an equivalence occurs in both polarities and is rejected as well. For example, `PRV004` Skolemizes $\forall X\,\forall Z\,\exists Y\,\forall W\; r(X,Z,Y,W)$ with the term $sK0(X,Z)$. The checker computes that Y lies in the scope of X and Z but not of W, compares this with the annotation and checks the replaced formula against the conclusion. Instantiation steps with `herbrand` symbols need no freshness condition, because an instance of a universal formula is entailed by it, and they are checked like other `thm` steps.

5.5 Delegation, Verdicts and Soundness

Delegation to an external ATP. Steps without a rule-specific checker are validated by an external ATP, in line with the competition rules. Only steps with status `thm` or `cth` are delegated, and the ATP receives exactly the premises of the step as axioms and its formula as conjecture, never the original conjecture or the rest of the proof. The answers `Theorem`, `Unsatisfiable` and `ContradictoryAxioms` confirm the step, `CounterSatisfiable` and `Satisfiable` refute it and every other answer, a timeout, or a missing prover leaves it *unresolved*. For the junction rules, a failed shape check is not final, and the step is passed to the ATP, because their names are also used for unusual but valid variants. If the ATP does not confirm such a step, the original issue is reported. Two sound shortcuts avoid prover calls: a final `consequence` step to `$false` with complementary parents, and an `add_hypothesis` step of the form $H \Rightarrow \varphi$ with parent φ.

Verdicts and soundness. Each step ends as *valid*, *invalid*, or *unresolved*. The verdict of the proof is *failed* if any step is invalid, a global check fails, or the proof cannot be processed, *uncertain* if any step is unresolved or the time limit is exceeded, and *verified* otherwise (Table 6). The option `--binary-verdict` maps *uncertain* to *failed* for a two-valued output.

If the verdict is *verified*, every condition of Section 2.2 has been checked, every `thm` step has been shown to follow from its parents by a rule check or the ATP, and every Skolemization step satisfies the conditions above. The induction of Section 2.4 then shows that the conjecture follows from the axioms. The argument assumes that the rule checkers are correct and that the ATP is sound, which Section 8.2 discusses.

Table 6: Verdicts, printed SZS status and score under the ProoVer 2026 scheme.

Verdict	SZS output	Proof valid	Proof invalid
verified	`VerifiedGood`	+1	−10
failed	`VerifiedBad`	−1	+2
uncertain	`Unknown` or `Timeout`	0	0

Resource control and alternatives. Each proof is checked in its own worker process, which is terminated when the limit is exceeded. A hanging ATP call or a crash therefore cannot affect the verdict of another proof. Because all traversals of formulas are recursive and the benchmark contains deeply nested formulas, the worker uses a larger stack.

The main alternatives were considered and rejected for the ProoVer setting. Sending every step to an ATP would make the verifier simpler, but slower, dependent on the ATP for every proof and unable to check `esa` steps at all. A single generic entailment test instead of rule-local checkers could neither explain failures nor reject steps that follow by the wrong rule. A binary verdict would score two points more on this benchmark, but loses more when the prover is missing or slow (Section 7.6).

6 Implementation

This chapter describes how the design is implemented in Python and points out the details that matter for soundness.

6.1 Overview and Parser

TSTPy is written in Python 3.12 and uses the parser library `lark` as its only runtime dependency. It consists of about 5 500 lines in four layers, namely the parser and the syntax tree, the rule checkers with shared utilities, the driver `main.py` and the tooling for scoring and measurement (Table 9 lists the checker modules). The checkers are pure functions of a step and its parents. They do not print, read files, or modify the syntax tree, and they do not import each other or the driver. Each of them can therefore be loaded and tested on its own.

Parser and data model. The parser is generated by Lark from a grammar for the required TPTP subset using the LALR algorithm with cached tables. A transformer maps the grammar productions to Python data classes for terms, atoms, equalities, negations, binary formulas, n-ary junctions and quantified formulas, and normalizes some constructs, for instance `<=` into an implication. A proof step is an annotated formula with its source or inference record, and roles, statuses and rule names are `StrEnum` classes, which makes the set of recognized values explicit. An own grammar was used instead of reusing an existing TPTP parser, because the verifier needs full access to the annotations `new_symbols` and `skolemize` and to n-ary junctions, and extending a general-purpose parser for this would have been about as much work.

6.2 Driver and Checkers

The function `check_proof_file` parses both files, builds an index from names to steps, runs the global checks and then visits the steps in file order. It selects the checker in an `if`/`elif` chain by role and rule name. Every formula with a `file(...)` source goes to the provenance check, the negated conjecture to its own checker, the rules of Table 5 to their modules and every other step with status `thm` or `cth` to the ATP. Steps with another status and no checker, and formulas with an `introduced(...)` source, are reported as undecidable. The driver does not stop at the first error but collects the findings in one map per rule family, from which `classify_verdict` computes the verdict. It also keeps the function symbols of the problem and the Skolem symbols introduced so far, because freshness is the one condition that cannot be decided locally by the Skolem checker.

Alpha-equivalence is implemented as a simultaneous traversal of both formulas with two mappings between their bound variables, after consecutive quantifiers of the same kind have been merged and nested junctions flattened. The dependency check uses an iterative depth-first search that marks each step as unvisited, in progress, or finished. An edge to a step that is still in progress closes a cycle, and because the search keeps its own stack instead of recursing, its depth does not depend on the length of the proof.

Rule checkers. Most rule checkers are short, and three details decide whether they are sound.

Capture check. `instantiate` and `existential_gen` use template matching, in which the quantified variables of the template may be mapped to any term, but each to the same term everywhere. The matching carries the variables bound inside the formula and refuses a term that contains one of them. Otherwise, instantiating x in $\forall x \exists y\, p(x,y)$ with y would yield $\exists y\, p(y,y)$, which does not follow.

Unification. Resolution and paramodulation rename the variables of the second parent apart before they unify, use an occurs check and resolve the unifier into idempotent form before they apply it. Without these steps, unrelated variables with the same name would constrain each other, and conclusions that are too strong would be accepted.

Polarity. The Skolem checker derives the polarity of each operand from its connective, including the antecedent of implications and the operands of NOR and NAND, renames bound variables apart before it compares the replaced formula with the conclusion, and reports a missing or repeated status.

The `rewrite` rule is checked most restrictively. In the benchmark format a rewrite step does not record the equation it uses. The transformation therefore cannot be reconstructed, and the checker only accepts a conclusion that equals the parent up to renaming. These steps are not delegated to the prover, because an entailment test would accept the invalid rewrite step of `PRV024` (Section 7.6).

Delegation to the external ATP. The function `validate_step_with_atp` writes the premises as axioms and the step as conjecture into a temporary TPTP file, quoting names that are not valid TPTP tokens, and starts Vampire in CASC mode as a subprocess with a time limit. It reads the SZS status line and returns *true*, *false*, or *unknown* according to Section 5.5. The prover can be replaced through the environment variables `EXTERNAL_ATP_COMMAND` and `EXTERNAL_ATP_BINARY`. The calls are sequential, and each may use the full limit of the proof.

6.3 Example of Skolemization in PRV008

The invalid proof PRV008 shows why the Skolem checker compares the replaced formula with the conclusion instead of testing an entailment. Its axiom

$$\forall x \exists y \left(s(x,y) \wedge \forall x\, t(x,y) \right)$$

uses the name x for an outer and an inner quantifier. A correct Skolemization replaces y by $sK0(x)$ and renames the inner variable, which gives

$$\forall x \left(s(x, sK0(x)) \wedge \forall x_1\, t(x_1, sK0(x)) \right),$$

in which the second occurrence of $sK0(x)$ still refers to the outer x. PRV008, on the other hand, states

$$\forall x \left(s(x, sK0(x)) \wedge \forall x\, t(x, sK0(x)) \right),$$

in which the inner quantifier captures the argument of the second Skolem term. The conclusion thus claims $t(x, sK0(x))$ for every x. The two formulas look almost the same, and the shadowing of x is the only difference. The checker substitutes $sK0(x)$ for y in the parent, renames the bound variables apart and finds that the result is not alpha-equivalent to the stated conclusion. It therefore reports the step `sk` as an incorrect Skolemization. All other checks pass, and the verdict is `VerifiedBad`. The stated formula follows from the correct Skolemization. A check that only asks whether the proof as a whole is plausible is unlikely to notice the defect. It is only visible to a check of the step against its rule.

6.4 Resource Control and Tests

Each proof is checked in a process created with `multiprocessing`. The parent reads the result from a queue and terminates the worker when the limit is exceeded. Inside the worker, checking runs in a thread with a stack of 64 MB and a recursion limit of 100 000, because the transformer and the checkers traverse formulas recursively and raising only the recursion limit would risk a crash of the interpreter. A prover started by the worker has its own time limit. It therefore ends shortly after the worker, even if the worker is terminated.

Tests and tooling. The test suite consists of 293 `pytest` tests in 26 files. The tests fall into four groups. Unit tests call a single checker on a few hand-written formulas. Regression tests reproduce every defect found during development with the smallest example that triggers it. Property-based tests modify correct proofs in small ways, for instance by swapping a connective or removing a premise, and check that the verifier detects the change. End-to-end tests run the driver on complete proofs. Two tests need an external prover, and all others run without one, which keeps the continuous integration independent of it. The command-line interface checks a single proof or a whole directory with scoring, and the options `--binary-verdict`, `--quiet` and `--debug` control the output. The script `scripts/eval_all.py`[1] runs all experiments of Chapter 7

[1]The repository is available at `https://gitlab.lrz.de/masters2026/master-thesis-proover` and `https://github.com/AlexanderGiller/master-thesis-proover`, see Appendix A.

and writes their tables and numbers as LaTeX files. No number in that chapter is typed by hand.

7 Evaluation

This chapter evaluates TSTPy against the research questions of Section 1.2.

7.1 Setup

Benchmarks. Of the 100 problem/proof pairs initially provided for ProoVer 2026, 91 (44 valid, 47 invalid) form the scored benchmark, which is called the *ProoVer set*. Their expected results come from the file `results.csv` of the organizers (Cailler & Guilloud 2026b). The remaining 9 deprecated pairs are only reported where this is stated. The *TST set* consists of 76 proofs written for this thesis to test individual checks (Table 7, results per proof in Table 17), five of which are edge cases with an open expected result and are not counted. The *Extra set* consists of 109 generated proofs for the rules that the TST set hardly covers (Section 7.2 and Table 18).

Table 7: Groups of the TST set.

Group	Proofs	Purpose
`ax`	11	Provenance: modified, renamed, or re-roled axioms and conjectures
`c`	13	Correct proofs with edge cases, such as renamed or unordered steps
`cp`	3	Conjecture used as premise, inconsistent axioms
`nc`	11	Negated conjecture: wrong status, parent, role, or formula
`sk`	25	Skolemization: arity, scope, freshness, polarity, wrong result
`st`	13	Structure: no `$false`, cycles, undefined premises, unknown status

Metrics and prover. The primary metric is the score under the ProoVer 2026 scheme with the default three-valued verdict. The verdict matrix, wrong acceptances, run time, memory and the share of steps decided without the prover are also reported. All delegated steps go to Vampire (Kovács & Voronkov 2013) in CASC mode, the strongest first-order prover of recent CASC competitions (Sutcliffe 2025). Because every delegated step needs a confirmation of the prover, the strength of the prover determines how many valid proofs can be verified, which is why the strongest system was chosen instead of comparing several.

Environment. All measurements were made on a laptop with 12th Gen Intel(R) Core(TM) i7-1255U (12 threads, 8 GB RAM) under WSL 2, with Python 3.12.3, `lark` 1.3.1, Vampire 5.0.1 (Release build, commit 1b13eaf on 2026-01-18 12:14:50 +0000)and a limit of 30 s per proof, on 2026-10-23 with commit 10bfb54. Each proof was checked 3 times in a fresh process, and run times are medians. As in the competition, a run includes the start-up of the interpreter and ends with the verdict.

7.2 Correctness and Score

Table 8 shows the verdicts on the ProoVer set, and the file `results/eval/per_proof.csv` of the repository lists the result for every proof (Appendix C). TSTPy accepts all 44 valid proofs, rejects 46 of the 47 invalid proofs and answers *unknown* for one, which gives 136 of 138 points (98.6 %) without a wrong acceptance. The two missing points stem from `PRV097`, which contains a step with the rule `trivial` and status `esa`. There is no internal checker for it, and the step must not be delegated, because the status `esa` does not allow **it** The verifier therefore abstains (Section 7.7). With the binary verdict this proof is rejected, and the score is the maximum of 138 points. On the deprecated proofs, too, all verdicts agree with the expected results. Two invalid proofs, `PRV091` and the deprecated `PRV057`, are rejected by the parser, because they use names starting with an upper-case letter as a Skolem symbol or a step name. Such names are variables in TPTP. The proofs are therefore not well-formed, which is their intended defect.

Table 8: Verdicts on the 91 pairs of the ProoVer set and resulting score. The last row also includes the 9 deprecated proofs.

expected \ verdict	verified	failed	unknown	timeout	points
valid (44)	44	0	0	0	+44
invalid (47)	0	46	1	0	+92
total (91 official pairs)	44	46	1	0	+136
incl. 9 deprecated (100)	47	52	1	0	+151

On the TST set the verdicts agree with the expectation for 70 of 71 proofs (Table 17). The exception, `TST043`, derives `$false` from the axioms and the negated conjecture in a single step. The verifier accepts it, because the step is a valid inference, although the test marks it as incorrect for its coarse granularity. Whether such a step is acceptable is a question of the competition rules rather than of sound reasoning.

Rule-level test suite. The TST set concentrates on provenance, the negated conjecture and Skolemization, where competition proofs are most likely to contain errors. For the other rules the Extra set was generated. For each of 20 rules, it contains a proof that applies the rule correctly and several variants with a specific defect, such as a wrong premise, an inverted conclusion, a missing side condition, or a captured variable, plus 8 sound but unusual edge cases. The verifier accepts all 35 valid proofs and rejects all 66 invalid ones (Table 18), and in every case the rule checker itself reports the error, not only the final `$false` step. Of the edge cases, 2 are accepted and 6 rejected, which was regarded as cautious behavior on formulations that no competition proof uses. In earlier versions, this suite exposed defects that the ProoVer set could not reveal, because it contains no proof that exercises them.

7.3 Rule Coverage and Modularity (RQ1, RQ2)

The ProoVer set contains 15573 derived steps of 34 rules.[2] Of these steps, 15449 (99.2 %) are decided by an internal checker, 123 by the prover alone and 1 by the prover after a failed shape

[2] `PRV091` is not counted, because the parser rejects it.

check. The aggregate is dominated by three proofs with 10 005 steps each that consist almost entirely of `modus_ponens` and `skolemize`. In half of the proofs, at least 50 % of the steps are decided internally.

Figure 4 shows the rules by the number of proofs that use them. `consequence` is the most frequent rule (87 of 91 proofs), followed by `skolemize` (41) and `instantiate` (29). Apart from `consequence`, which has no syntactic criterion, every rule that occurs in more than five proofs has an internal checker. Even for `consequence`, 59 of the 88 steps have complementary parents and are decided by the shortcut of Section 5.5 without a prover call. Of the 14 rules without one, only `consequence` and `add_hypothesis` occur in more than four proofs, and most of the others in a single one. In Figure 4 these single-proof rules are grouped. `duplicate`, `rewrite`, `transitivity` are checked internally, and `alpha`, `case_analysis`, `contrapositive`, `de_morgan`, `disjunctive_syllogism`, `over_generalize`, `reassociate`, `simplify`, `tautology` are decided by the prover. In answer to RQ1, a verifier that covers Skolemization, instantiation, generalization, the propositional rules and the structural rules checks the bulk of the benchmark itself, and delegation handles the long tail of rules, for which an own checker would add trusted code for a handful of steps.

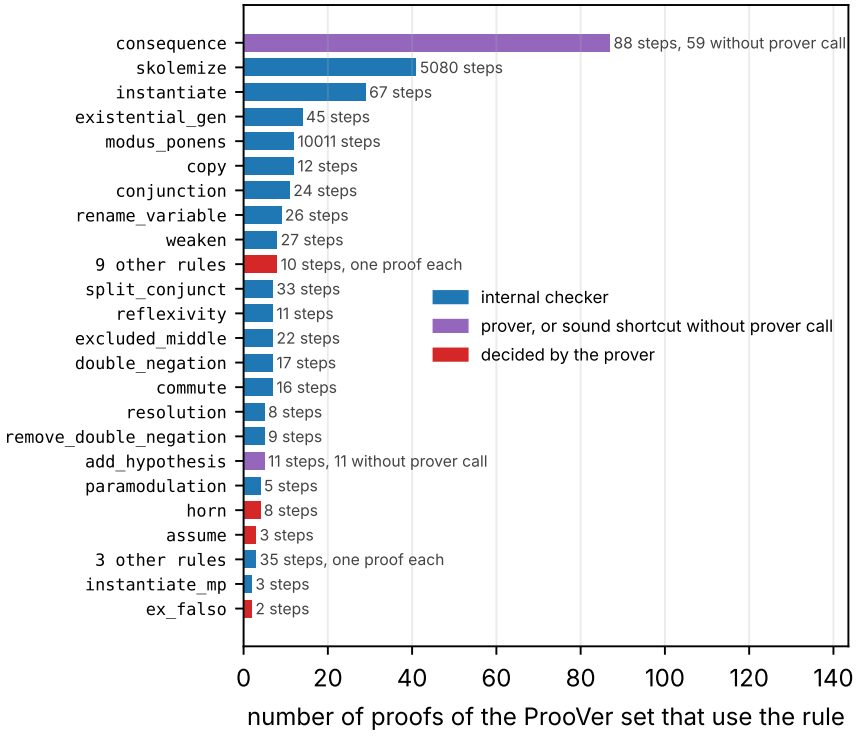


Figure 4: Inference rules of the ProoVer set by the number of proofs that use them. Blue: internal checker. Red: decided by the prover. Purple: delegated, but some steps are decided by a sound shortcut. Rules used in only one proof are grouped by color. The labels give the number of steps.

Architecture metrics. Table 9 gives the size and the statement coverage of each checker module. The checker package has 2856 lines, from 74 to 619 per module, and the tests cover 87.2 % of its statements and 90.4 % of the whole `src` directory. Because each module can be tested alone, the

rule-level tests call the checkers directly with a few formulas, without parsing a proof or starting a prover. Adding a rule needs a new checker function and a new branch in the driver.

Table 9: Size and statement coverage of the checker modules (measured with `pytest-cov`).

Module	Lines	Statements	Coverage
`skolem_checker.py`	619	266	82.7 %
`clausal_checker.py`	500	310	84.5 %
`formula_utils.py`	270	144	81.2 %
`negated_conjecture_checker.py`	263	114	90.4 %
`external_atp_checker.py`	197	119	84.9 %
`junction_checker.py`	171	76	92.1 %
`propositional_checker.py`	161	72	87.5 %
`alpha_eq.py`	155	85	97.6 %
`instantiate_checker.py`	129	67	88.1 %
`dependency_checker.py`	125	61	100.0 %
`structural_checker.py`	98	53	94.3 %
`file_dir.py`	94	44	95.5 %
`existential_gen_checker.py`	74	36	86.1 %
Checker package	2856	1447	87.2 %
All of `src`	4252	2195	90.4 %

7.4 Run Time and Memory (RQ3)

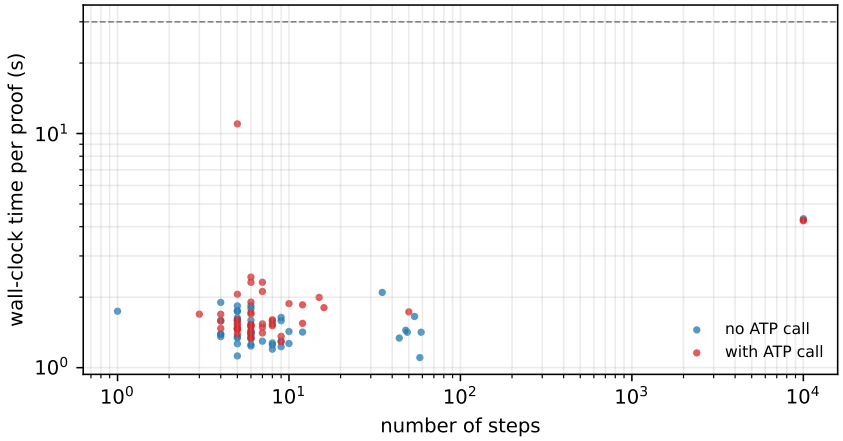


Figure 5: Wall-clock time per proof of the ProoVer set against the number of steps. Proofs with a prover call are drawn in red, and the dashed line is the time limit.

Figure 5 plots the run time of each proof against its number of steps. The median is 1.52 s, the 90th percentile 2.06 sand the maximum 11.00 s, and the peak memory is 26 MB at the median and 172 MB at the maximum. About 1.40 s of a typical run are the start-up of the interpreter, which is

slow under WSL 2. The checks themselves are cheap. Table 10 shows where the time goes for the slowest proofs. The three proofs with 10 005 steps need about 4.3 s each, most of it for start-up and parsing (2.6 to 2.9 s), and less than 0.2 s for the rule checks.

Table 10: The six slowest proofs of the ProoVer set. "Interpreter" is the time from the start of the process until the verifier begins to check, that is, the start of the Python interpreter and the import of the modules. All phases come from the median repetition and add up to the total, and "MB" is the peak resident memory.

Proof	Steps	ATP calls	Interpreter	Parsing	ATP	Rule checks	Total	MB
`PRV099+1.s`	5	2	1.75 s	94 ms	9.12 s	33 ms	11.00 s	172
`PRV072+1.s`	10005	0	1.39 s	2.88 s	0.0 ms	49 ms	4.33 s	96
`PRV073+1.s`	10005	1	1.43 s	2.62 s	74 ms	152 ms	4.27 s	93
`PRV071+1.s`	10005	1	1.41 s	2.56 s	75 ms	197 ms	4.24 s	93
`PRV093+1.s`	6	1	2.26 s	23 ms	125 ms	26 ms	2.44 s	59
`PRV092+1.s`	7	1	2.12 s	38 ms	133 ms	23 ms	2.32 s	59

The 54 prover calls take 8.6 % of the total time, with a median of 80 ms. Only 1 proof needs more than 10 s, namely `PRV099`, which spends 9.0 s in a single prover call (Section 7.7). All other calls take less than 0.3 s.

7.5 Robustness (RQ3)

Mutated proofs. To generate mutated proofs, derived steps of 43 valid proofs with at most 80 steps were mutated by changing a connective, swapping the sides of an implication, swapping a quantifier, negating an atom, removing or replacing a premise, or changing the status `thm` to `cth`, at most two steps per proof and operator (Table 11). A mutant that the verifier accepts is not necessarily a miss, because many mutations preserve validity. Every accepted mutant was therefore checked with Vampire. Of the 361 mutants, 303 are rejected, and all 58 accepted ones are still valid. Every mutant that is no longer valid is thus detected. Most accepted mutants (41 of 58) change the status to `cth` of a step that still follows from its premises, which the verifier correctly treats as informational. The others swap the operands of a symmetric connective or cite another premise from which the conclusion also follows. Because this classification only tests entailment, it cannot judge defects such as a dropped Skolem dependency, which the TST group `sk` covers instead.

Malformed input. Table 12 shows ten kinds of broken input derived from `PRV010+1.s`. TSTPy never crashes or hangs, 3 inputs end in a parser error, which counts as a rejection, and the others are rejected by the checks. The only accepted input replaces a rule name by an unknown one. As required in Chapter 4, such a step is passed to the prover like every other step without a checker. Here the prover confirms it, because the step is in fact entailed by its premises.

Missing prover. Running the ProoVer set with a prover command that does not exist (Table 13) accepts no invalid proof, which confirms the fail-closed policy. Although `consequence` occurs in 87 proofs (Figure 4), only 27 proofs become *Unknown*, for two reasons. First, 22 of the 44 valid

Table 11: Mutation operators and outcomes. Every accepted mutant was checked with Vampire and is still valid. The detection rate is therefore the share of rejected mutants among those that are no longer valid.

Mutation operator	Mutants	Rejected	Accepted, still valid	Detection rate
Change connective ($\wedge \leftrightarrow \vee$)	21	19	2	100.0 %
Swap sides of an implication	21	18	3	100.0 %
Swap quantifier kind ($\forall \leftrightarrow \exists$)	20	19	1	100.0 %
Negate an atom	77	76	1	100.0 %
Remove a premise	68	66	2	100.0 %
Cite another premise	77	69	8	100.0 %
Change status `thm` to `cth`	77	36	41	100.0 %
Total	361	303	58	100.0 %

Table 12: Outcome for malformed inputs. `error` means that the parser rejected the file. Every input is processed in less than 0.2 s.

Input	Outcome	Input	Outcome
empty file	`VerifiedBad`	unknown status	`error`
truncated in the middle of a formula	`error`	duplicate step names	`VerifiedBad`
binary garbage	`error`	deeply nested negations (3000)	`VerifiedBad`
unknown inference rule	`VerifiedGood`	deeply nested terms (3000)	`VerifiedBad`
missing inference annotation	`VerifiedBad`	only comments	`VerifiedBad`

proofs need no prover at all, because their final step has complementary parents and is decided by the shortcut, or because they end with an internally checked resolution step. The other 22 valid proofs become *Unknown*. Second, an invalid proof is rejected as soon as any check finds a defect, whatever happens to its other steps. For 42 of the 47 invalid proofs the defect lies in an internally checked step, and only 5 have their defect in a step that needs the prover. With the binary verdict the 22 undecided valid proofs are rejected instead, which costs one point each, whereas the three-valued verdict abstains on them.

Table 13: The ProoVer set with and without the external prover, under the three-valued and the binary verdict. No setting verifies an invalid proof, and the binary verdict has no *Unknown*.

		valid proofs		invalid proofs		
prover	verdict	verified	rejected	rejected	unknown	score
with prover	three-valued	44	0	46	1	+136
	binary	44	0	47	–	+138
without prover	three-valued	22	0	42	27	+106
	binary	22	22	47	–	+94

7.6 Ablations

Internal checkers against the prover. To see what the internal checkers add, up to eight internally checked steps per proof of the ProoVer set (proofs up to 400 steps) were sampled and Vampire was asked with a limit of 10 s whether they follow from their premises. Of the 153 steps, the prover confirms 144 that the checker accepts, and it refutes none of them. For 3 accepted steps, a `rename_variable` step in `PRV008`, a `conjunction` step in `PRV012` and an `instantiate_mp` step in `PRV018`, Vampire does not finish within 10 s, although the steps are trivial for the internal checker. On the other hand, it confirms 2 steps that the checker rejects, a `modus_ponens` step in `PRV023` and a `rewrite` step in `PRV024`. Their conclusions follow from the premises, but not by the cited rule. The internal checks are therefore faster on hard-looking trivial steps and stricter than entailment, which is intended, because the task is to verify the stated inference.

Three-valued verdict. Table 13 also compares the two verdict policies. With the prover, the binary verdict scores two points more, because it rejects `PRV097` instead of abstaining. Without the prover the order is reversed, 106 against 94 points, because 27 proofs are undecided and a rejected valid proof costs a point, whereas an abstention costs nothing. The three-valued verdict was chosen as the default, because it limits the loss when the prover is missing or slow and distinguishes incorrect steps from steps that could not be checked.

7.7 Remaining Errors and Threats to Validity

The only suboptimal verdict is the abstention on `PRV097`. A checker for `trivial` with status `esa` would turn it into a verdict, but the benchmark contains only one such step, which is not enough to derive the intended semantics with confidence, and a guess could cause a wrong acceptance.

`PRV099` is the only proof whose run time is dominated by the prover. Its step `hard` derives `$false` from the negated conjecture alone, which is the pigeonhole formula for ten pigeons and nine holes with 90 variables and 415 short clauses. Every resolution refutation of this formula has exponential size (Haken 1985). Vampire therefore needs 9.0 s although the formula is small. After it, the proof contains a second step `badcheap` that derives `$false` from the satisfiable axiom $g(e)$ and is refuted in less than 0.1 s. The proof is presumably designed to test whether a checker spends its time on the hard step before it reaches the cheap defect. The verifier returns the correct verdict, but it falls into this trap, because it checks the steps in file order and calls the prover for `hard` before it reaches `badcheap`. Checking cheap steps first or running prover calls in parallel with a short first limit would avoid this and is left to future work (Section 9). In an earlier run on a two-core virtual machine with Vampire 5.1.0, the call for `hard` exceeded the limit and the proof counted as an abstention, which would have cost two points.

Threats to validity. The verifier was developed against the ProoVer sample proofs. The ProoVer set is therefore not an independent test set, and the TST and Extra sets reflect the author's understanding of the rules. The mutation and missing-prover experiments do not depend on the benchmark, but they rely on Vampire to classify accepted mutants. This classification is only as reliable as the prover. The benchmark comes from a fixed set of generators, and the results do not transfer to other provers (Section 8.3). The competition runs each checker on one of two Intel Xeon E5-2620 v4 CPUs with 2.1 GHz and 128 GiB memory. A single core of this server processor is slower than a

core of the laptop used here, and the verifier uses only one core. Assuming a factor of 1.5 to 2.5, which is an estimate and not a measurement, the median would rise to two to four seconds and the large proofs to 6 to 11 s, while `PRV099` could reach 16 to 28 s and come close to the limit. A timeout there would turn the rejection into an abstention, but could not lead to a wrong acceptance.

8 Discussion

This chapter interprets the results, discusses what TSTPy guarantees, examines how far it transfers to the proofs of an automated prover and lists its limitations.

8.1 Interpretation of the Results

TSTPy loses only two points on the benchmark and accepts no invalid proof, which matters more than the score, because a single wrong acceptance costs as much as ten correct ones. This does not show that the verifier is complete. The benchmark was available during development, and the invalid proofs contain the kinds of defects the checks were built for. The Extra set found defects that the benchmark could not, and if the verifier loses points on unseen proofs, the fail-closed policy makes it lose them in the cheap direction. The three-valued verdict costs two points here, but limits the loss when the prover is missing or slow.

The run times are not dominated by the prover, whose calls take 8.6 % of the total time, but by the start-up of the interpreter. The order "cheap checks first" is therefore mainly a matter of precision and independence from the prover. It also explains why steps that Vampire cannot confirm within 10 s, but that are trivial for an internal checker, do not endanger the verdict.

8.2 Trusted Computing Base and Design Trade-offs

The soundness argument of Section 5.5 holds if the parser, the alpha-equivalence check, the rule checkers, the serialization of formulas for the prover, the verdict aggregation and the prover itself are correct. A verifier that sends every step to an ATP has less own code, but it relies on the ATP, a saturation prover with a far larger code base, even for steps as simple as an instantiation. The internal checks of TSTPy replace the ATP for such steps. For most steps the trusted code is therefore the smaller one, about 2856 lines of checkers plus the parser and shared utilities, each of which can be read and tested rule by rule. Measured against the de Bruijn criterion, this is a partial success. The internal code is still an order of magnitude larger than an LCF kernel such as that of HOL Light (Harrison 2009), and every proof with a step that only the prover can decide also depends on Vampire. The competition rules explicitly allow this compromise, since they permit external provers for rules without a fixed specification. A stricter verifier could still use external provers, but would require them to produce a certificate for every delegated step that a small trusted kernel checks, as GDV-LP does with LambdaPi (Sutcliffe et al. 2025). The correctness of the checkers is supported by tests, the mutation experiment and the benchmark, but not proved.

Design trade-offs. A proof that derives `$false` in one step from the axioms and the negated conjecture, such as `TST043`, is delegated completely, and the verifier is then exactly as strong as the prover. Because each rule has its own checker, every rule can be tested separately and every

error can be traced to a single step. Some checks still need information about the whole proof, such as whether a Skolem symbol is fresh, and these are done in the driver. The driver also still selects the checker for each step in a fixed chain of conditions instead of a modular registry. Python favored readability and fast development, which was valuable for a thesis with many rules, and is fast enough for this task. Only for the very large proofs does the pure Python cost matter, and there mainly in the parser. Process isolation makes timeouts and crashes harmless, but adds a start-up cost per proof and makes it harder to share state such as a cache of prover results.

8.3 Transfer to Other Provers: A Case Study with Vampire

To see how much of the design survives when the proofs come from an automated prover, Vampire 5.0.1 was run in CASC mode for 30 s on the 176 problem files of the ProoVer set, the deprecated pairs and the TST set. It found 142 proofs (139 *Theorem*, 3 *ContradictoryAxioms*), and Table 14 shows what happens when they are given to TSTPy unchanged.

Table 14: Verifier on unmodified Vampire proofs.

Outcome	Proofs
Rejected by the parser (`introduced(...)` annotations)	66
Parsed, verdict *failed*	75
Parsed, timeout	1
Parsed, verdict *verified*	0

No proof is verified, for three reasons. *Syntax*: Vampire records Skolem symbols as separate `introduced(definition,...)` formulas and cites input formulas as `file(...,unknown)`, which a normalizing front end could handle. *Vocabulary*: the 142 proofs use 22 inference names, the 176 reference proofs of these sets 39, and only `resolution` and `negated_conjecture` occur in both, with different meanings. Most of the 22 153 inferences belong to clausification and splitting, for example `cnf_transformation` and `avatar_*`. *Status*: in Vampire's output only `negated_conjecture` and `skolemisation` carry a status. Under the fail-closed policy no other step may therefore be delegated.

The first experiment mostly measures format differences. To test whether the *checks* carry over, the 5 141 Skolemization steps of the 176 reference proofs, at most ten per proof and mutation kind, were taken, mutants with a known defect were derived and the internal Skolem check was compared with a *Vampire entailment check* that asks Vampire whether the conclusion C entails the parent P (Table 15). This is the only entailment that holds for a correct step, and it is what a verifier built around a generic prover would test.

Defects that change the formula are found by both, but the defect specific to Skolemization, a missing dependency, is found by the Vampire entailment check in none of the 25 cases, because a Skolem term with too few arguments makes C stronger and $C \models P$ still holds. Of the 35 flawed steps in the reference proofs, which include wrong annotations and `thm` instead of `esa`, the Vampire entailment check rejects only 11. The one control step rejected by the Vampire entailment check is a Skolemization in the antecedent of an implication, which the internal check of the measured version also wrongly accepted and which the final version rejects.

Table 15: Internal Skolem check and Vampire entailment check ($C \models P$) on correct, mutated and flawed Skolemization steps. The numbers are the steps that each check rejects. A perfect check rejects every mutated and flawed step and no control step.

		rejected by	
Step kind	Steps	Skolem check	Vampire entailment check
unchanged (control)	116	0	1
missing dependency	25	25	0
universal variable eliminated	56	56	26
negated atom	108	108	106
dropped conjunct	11	11	5
flawed steps of the reference proofs	35	35	11

A verifier that performs well on the competition set therefore does not automatically transfer to prover output. This is expected, because the benchmark is designed to test rule-level checks on a controlled vocabulary. TSTP fixes the syntax of derivations, but not a common set of inference rules and their semantics. Every prover therefore names and structures its steps in its own way (Sutcliffe 2026d). One purpose of a competition such as ProoVer is to establish such a common ground, and a uniform rule vocabulary is what makes verifiers comparable in the first place. The gap to the output of Vampire is therefore a consequence of this early stage of standardization rather than a weakness of the benchmark or of the verifier. The checkers are made for the rules of the benchmark, which resemble natural deduction and change a formula only a little at a time. A proof from a prover looks different and consists mainly of clausification, splitting and saturation steps. The invalid proofs of the benchmark contain deliberately chosen defects, whereas prover proofs would be wrong for other reasons. Satisfiability-preserving steps such as Skolemization, definitions and AVATAR splitting cannot be validated by an entailment test. A prover that checks equisatisfiability could validate them in principle, but delegating Vampire's own steps to Vampire would make the trusted base circular. A direct extension would need a normalizing front end, native checkers for clausification and splitting and an argument for satisfiability-preserving steps, as certificate-based approaches provide (Wetzler et al. 2014; Andreotti et al. 2023).

8.4 Limitations

- **Scope.** Only FOF problems and the rule vocabulary of the ProoVer 2026 benchmarks are supported. Other rules are delegated or lead to abstention.
- **Weak rule checks.** `rewrite` steps omit their equation. Only conclusions equal to the parent are therefore accepted, which is sound but incomplete. Herbrand annotations are parsed but not checked separately.
- **Rules without semantics.** `trivial` with status `esa` and any other unknown rule with a status other than `thm` or `cth` lead to abstention.
- **Dispatch.** The checkers are selected by an `if`/`elif` chain instead of the registry of the design. A new rule therefore needs a new branch in the driver.

- **Prover calls.** The calls are sequential, and each may use the full limit of the proof. One hard step can therefore use up the time of the whole proof, as in `PRV099`.
- **Evidence.** The benchmark sets are large enough to find defects, but not to estimate rare failure rates, and the correctness of the checkers is supported by tests, not proved.

9 Conclusion and Future Work

This thesis designed and implemented TSTPy, a Python-based verifier for TSTP derivations for ProoVer 2026. Starting from the asymmetric scoring scheme, it uses a fail-closed, three-valued verdict, checks most rules and Skolemization internally and delegates the remaining steps to Vampire under the restrictions of the competition.

RQ1. The ProoVer set uses 34 rules, of which 20 are checked internally, covering 99.2 % of the derived steps and at least 50 % in half of the proofs. The delegated rules are mainly the generic `consequence` and a long tail of rare rules. Skolemization is checked internally, as the competition requires, and an entailment test could not replace this check, because a step with too few Skolem arguments still entails its parent. Its checks for scope, polarity and freshness are what rejects many of the invalid proofs.

RQ2. Every rule check is a small function of a step and its parents that can be tested alone, which the 87.2 % statement coverage of the checker package and the rule-level test suite reflect. For most steps the trusted code is this checker package of 2856 lines rather than an external prover. A new rule still needs a branch in the driver, because the registry of the design is not implemented.

RQ3. On the 91 proofs of the benchmark TSTPy reaches 136 of 138 points without a wrong acceptance, with a median run time of 1.52 s and at most 172 MB of memory. Every mutant that is no longer valid is rejected, and without the prover no invalid proof is accepted. The run time is dominated by the start-up of the interpreter and, for very large proofs, by parsing. The exception is `PRV099`, whose pigeonhole step takes about 9 s in the prover.

Future work. That TSTPy does not yet handle unmodified Vampire proofs is to be expected, because a common vocabulary of inference rules is only now being established, and ProoVer is a step in this direction. The limitations of Section 8.4 suggest several directions.

- A registry-based dispatch in which a new rule needs only one entry and one function.
- Prover calls in parallel and with a short first limit. Cheap defects such as the one in `PRV099` would then be found before a hard step uses up the budget.
- A model finder that turns unresolved steps into defects, as in Nörgler.
- A smaller trusted base, for example by requiring the external prover to produce certificates that a small kernel checks.
- Support for CNF proofs and the rule vocabularies of other provers.

A Reproduction

The source code, the proofs, the raw results and the scripts are in the repository of the thesis, which is available at `https://gitlab.lrz.de/masters2026/master-thesis-proover` and mirrored at `https://github.com/AlexanderGiller/master-thesis-proover`, and the commit used for the measurements is 10bfb54. The verifier needs Python 3.11 or newer and `lark`, and the external prover Vampire is found in the search path or configured with `EXTERNAL_ATP_BINARY` or `EXTERNAL_ATP_COMMAND`.

```
python -m pip install -r requirements.txt -r requirements-dev.txt
python main.py ProoVer2026/PRV/PRV010+1.s        # one proof
python main.py -d ProoVer2026/PRV                # directory with scoring
python main.py --quiet ProoVer2026/PRV/PRV010+1.s
pytest
python scripts/eval_all.py --parts all --reps 3
```

Table 16: Command-line options of TSTPy.

Option	Meaning
`-p`, `--problem`	Problem file, by default `Problems/<name>.p` next to the proof
`-t`, `--timeout`	Wall-clock limit per proof in seconds (default 30)
`-d`, `--dir`, `-e`, `--prefix`	Batch mode: directory, file with expected results, prefix of the checked files
`--binary-verdict`	Print `VerifiedBad` instead of `Unknown`
`--quiet`, `--debug`	Print only the SZS status per proof, or additionally the dispatch of every step

The evaluation driver writes the raw data, including the verdict and run time of every proof, to `results/eval` and the tables and numbers of Chapter 7 to `thesis/generated`.

B Example Report

For every proof, the verifier prints one block per check and ends with the SZS status. The following output is the shortened report for the invalid proof `PRV002+1`, whose negated conjecture is a renamed copy of the conjecture instead of its negation. Most blocks without a finding are omitted, and the formulas are abbreviated.

```
Checking proof: ProoVer2026/PRV/PRV002+1.s
Against problem: ProoVer2026/PRV/Problems/PRV002+1.p

[OK] No circular dependencies detected
[OK] All axioms verified correctly
[OK] All conjectures verified correctly

[FAIL] Negated Conjecture Issues:
  - neg: formula is not the correct negation of parent
    conjecture; expected one of: [~![X]: ?[Y]: ..., ...],
    got: ![A]: ?[B]: ![C]: (p(A, B, C) | ...)

[OK] All skolem steps verified correctly

% SZS status VerifiedBad
```

C Results per Proof

The results of the measured run (Section 7.1) for every proof of all sets, including the nine deprecated pairs, are in the file `results/eval/per_proof.csv` of the repository, with the expected result, the verdict, the number of steps and prover calls, the prover time, the median wall-clock time and the peak memory of each proof. For the ProoVer set, Table 8 summarizes them. Tables 17 and 18 list the TST set and the Extra set, which are not described elsewhere proof by proof. The time is the median wall-clock time of three runs, including the start-up of the interpreter. The column "Result" compares the status with the expected result, and "abstain" denotes `Unknown` or `Timeout`, which score zero. The status `Error` marks proofs that the parser rejects, for which the verifier prints `VerifiedBad`. "Edge case" marks the proofs with an open expected result, five in the TST set and 8 in the Extra set, which are not counted. The only disagreement in the TST set, `TST043`, is discussed in Section 7.2.

Table 17: Results for each proof of the TST set (own test proofs).

Proof	Expected	Status	Result	Time (s)	ATP calls
TST001+1	correct	VerifiedGood	ok	1.77	0
TST002+1	correct	VerifiedGood	ok	1.57	0
TST003+1	correct	VerifiedGood	ok	1.52	0
TST004+1	correct	VerifiedGood	ok	1.31	0
TST005+1	correct	VerifiedGood	ok	1.29	0
TST006+1	correct	VerifiedGood	ok	1.29	0
TST007+1	correct	VerifiedGood	ok	1.29	0
TST008+1	incorrect	VerifiedBad	ok	1.40	0
TST009+1	incorrect	VerifiedBad	ok	1.61	0
TST010+1	incorrect	VerifiedBad	ok	1.50	0
TST011+1	incorrect	VerifiedBad	ok	1.41	0
TST012+1	incorrect	VerifiedBad	ok	1.46	0
TST013+1	incorrect	VerifiedBad	ok	1.45	0

Proof	Expected	Status	Result	Time (s)	ATP calls
TST014+1	incorrect	VerifiedBad	ok	1.70	0
TST015+1	incorrect	VerifiedBad	ok	1.51	0
TST016+1	incorrect	VerifiedBad	ok	1.41	0
TST017+1	incorrect	VerifiedBad	ok	1.44	0
TST018+1	incorrect	VerifiedBad	ok	1.71	0
TST019+1	incorrect	VerifiedBad	ok	1.37	0
TST020+1	incorrect	Unknown	abstain	1.65	0
TST021+1	incorrect	VerifiedBad	ok	1.49	0
TST022+1	incorrect	VerifiedBad	ok	1.73	0
TST023+1	incorrect	VerifiedBad	ok	1.69	1
TST024+1	incorrect	VerifiedBad	ok	1.73	1
TST025+1	incorrect	VerifiedBad	ok	1.78	0
TST026+1	incorrect	VerifiedBad	ok	1.87	1
TST027+1	incorrect	VerifiedBad	ok	1.69	0
TST028+1	incorrect	VerifiedBad	ok	1.44	0
TST029+1	incorrect	VerifiedBad	ok	1.61	0
TST030+1	incorrect	VerifiedBad	ok	1.44	0
TST031+1	incorrect	VerifiedBad	ok	1.66	0
TST032+1	incorrect	VerifiedBad	ok	1.56	0
TST033+1	incorrect	VerifiedBad	ok	1.60	0
TST034+1	incorrect	VerifiedBad	ok	1.58	0
TST035+1	incorrect	VerifiedBad	ok	1.66	0
TST036+1	incorrect	VerifiedBad	ok	1.50	0
TST037+1	incorrect	VerifiedBad	ok	1.64	1
TST038+1	incorrect	VerifiedBad	ok	1.56	0
TST039+1	incorrect	VerifiedBad	ok	1.54	0
TST040+1	incorrect	Error	ok	1.70	0
TST041+1	incorrect	VerifiedBad	ok	1.49	0
TST042+1	incorrect	VerifiedBad	ok	1.93	0
TST043+1	incorrect	VerifiedGood	wrong accept	1.59	1
TST044+1	incorrect	VerifiedBad	ok	1.50	0
TST045+1	incorrect	VerifiedBad	ok	1.65	0
TST046+1	incorrect	VerifiedBad	ok	1.68	0
TST047+1	correct	VerifiedGood	ok	1.42	0
TST048+1	correct	VerifiedGood	ok	1.36	0
TST049+1	correct	VerifiedGood	ok	1.37	0
TST050+1	correct	VerifiedGood	ok	1.29	0
TST051+1	correct	VerifiedGood	ok	1.36	0
TST052+1	correct	VerifiedGood	ok	1.53	1
TST053+1	incorrect	VerifiedBad	ok	1.44	0
TST054+1	incorrect	VerifiedBad	ok	1.42	0
TST055+1	incorrect	VerifiedBad	ok	1.47	0
TST056+1	incorrect	VerifiedBad	ok	1.43	0
TST057+1	incorrect	VerifiedBad	ok	1.42	0
TST058+1	incorrect	VerifiedBad	ok	1.50	0
TST059+1	incorrect	VerifiedBad	ok	1.57	0

Proof	Expected	Status	Result	Time (s)	ATP calls
TST060+1	incorrect	VerifiedBad	ok	1.42	0
TST061+1	incorrect	VerifiedBad	ok	1.46	0
TST062+1	incorrect	VerifiedBad	ok	1.30	0
TST063+1	incorrect	VerifiedBad	ok	1.32	0
TST064+1	incorrect	VerifiedBad	ok	1.41	0
TST065+1	incorrect	VerifiedBad	ok	1.46	1
TST066+1	incorrect	VerifiedBad	ok	1.45	0
TST067+1	incorrect	VerifiedBad	ok	1.52	0
TST068+1	edge	VerifiedGood	edge case	1.31	0
TST069+1	incorrect	VerifiedBad	ok	1.38	0
TST070+1	incorrect	VerifiedBad	ok	1.33	0
TST071+1	edge	VerifiedGood	edge case	1.79	0
TST072+1	incorrect	VerifiedBad	ok	1.61	0
TST073+1	edge	VerifiedGood	edge case	1.52	0
TST074+1	edge	VerifiedBad	edge case	1.44	0
TST075+1	incorrect	VerifiedBad	ok	1.52	1
TST076+1	edge	VerifiedBad	edge case	1.35	1

Table 18: Results for each proof of the Extra set (generated rule-level test proofs).

Proof	Expected	Status	Result	Time (s)	ATP calls
EXT001+1	correct	VerifiedGood	ok	1.45	0
EXT002+1	incorrect	VerifiedBad	ok	1.45	0
EXT003+1	incorrect	VerifiedBad	ok	1.44	0
EXT004+1	incorrect	VerifiedBad	ok	1.49	0
EXT005+1	incorrect	VerifiedBad	ok	1.44	0
EXT006+1	incorrect	VerifiedBad	ok	1.75	0
EXT007+1	correct	VerifiedGood	ok	1.48	0
EXT008+1	incorrect	VerifiedBad	ok	1.44	0
EXT009+1	edge	VerifiedBad	edge case	1.49	0
EXT010+1	correct	VerifiedGood	ok	1.61	0
EXT011+1	incorrect	VerifiedBad	ok	1.39	0
EXT012+1	incorrect	VerifiedBad	ok	1.45	0
EXT013+1	incorrect	VerifiedBad	ok	1.47	0
EXT014+1	incorrect	VerifiedBad	ok	1.46	0
EXT015+1	incorrect	VerifiedBad	ok	1.47	0
EXT016+1	incorrect	VerifiedBad	ok	1.47	0
EXT017+1	correct	VerifiedGood	ok	1.62	0
EXT018+1	incorrect	VerifiedBad	ok	1.44	0
EXT019+1	correct	VerifiedGood	ok	1.73	0
EXT020+1	incorrect	VerifiedBad	ok	1.67	0
EXT021+1	correct	VerifiedGood	ok	1.86	0
EXT022+1	incorrect	VerifiedBad	ok	2.20	0

Proof	Expected	Status	Result	Time (s)	ATP calls
EXT023+1	incorrect	VerifiedBad	ok	2.11	0
EXT024+1	incorrect	VerifiedBad	ok	2.07	0
EXT025+1	incorrect	VerifiedBad	ok	2.01	0
EXT026+1	correct	VerifiedGood	ok	2.05	0
EXT027+1	incorrect	VerifiedBad	ok	1.90	0
EXT028+1	incorrect	VerifiedBad	ok	1.75	0
EXT029+1	incorrect	VerifiedBad	ok	1.96	0
EXT030+1	incorrect	VerifiedBad	ok	1.81	0
EXT031+1	correct	VerifiedGood	ok	1.89	0
EXT032+1	incorrect	VerifiedBad	ok	1.57	0
EXT033+1	incorrect	VerifiedBad	ok	1.62	0
EXT034+1	correct	VerifiedGood	ok	1.37	0
EXT035+1	incorrect	VerifiedBad	ok	1.27	1
EXT036+1	incorrect	VerifiedBad	ok	1.34	1
EXT037+1	incorrect	VerifiedBad	ok	1.34	1
EXT038+1	edge	VerifiedGood	edge case	1.43	1
EXT039+1	correct	VerifiedGood	ok	1.17	0
EXT040+1	incorrect	VerifiedBad	ok	1.34	1
EXT041+1	incorrect	VerifiedBad	ok	1.55	1
EXT042+1	correct	VerifiedGood	ok	1.40	0
EXT043+1	incorrect	VerifiedBad	ok	1.54	1
EXT044+1	correct	VerifiedGood	ok	1.21	0
EXT045+1	incorrect	VerifiedBad	ok	1.16	1
EXT046+1	incorrect	VerifiedBad	ok	1.89	1
EXT047+1	incorrect	VerifiedBad	ok	1.86	1
EXT048+1	incorrect	VerifiedBad	ok	1.60	0
EXT049+1	correct	VerifiedGood	ok	1.55	0
EXT050+1	incorrect	VerifiedBad	ok	1.47	0
EXT051+1	incorrect	VerifiedBad	ok	1.58	0
EXT052+1	incorrect	VerifiedBad	ok	1.67	0
EXT053+1	correct	VerifiedGood	ok	1.81	0
EXT054+1	incorrect	VerifiedBad	ok	1.89	0
EXT055+1	correct	VerifiedGood	ok	2.18	0
EXT056+1	incorrect	VerifiedBad	ok	1.38	0
EXT057+1	correct	VerifiedGood	ok	1.37	0
EXT058+1	correct	VerifiedGood	ok	1.37	0
EXT059+1	incorrect	VerifiedBad	ok	1.33	0
EXT060+1	incorrect	VerifiedBad	ok	1.22	0
EXT061+1	correct	VerifiedGood	ok	1.46	0
EXT062+1	incorrect	VerifiedBad	ok	1.22	0
EXT063+1	incorrect	VerifiedBad	ok	1.32	0
EXT064+1	incorrect	VerifiedBad	ok	1.29	0
EXT065+1	correct	VerifiedGood	ok	1.26	0
EXT066+1	incorrect	VerifiedBad	ok	1.27	0
EXT067+1	correct	VerifiedGood	ok	1.33	0
EXT068+1	incorrect	VerifiedBad	ok	1.33	0

Proof	Expected	Status	Result	Time (s)	ATP calls
EXT069+1	incorrect	VerifiedBad	ok	1.17	0
EXT070+1	edge	VerifiedBad	edge case	1.38	0
EXT071+1	correct	VerifiedGood	ok	1.41	0
EXT072+1	incorrect	VerifiedBad	ok	1.43	0
EXT073+1	incorrect	VerifiedBad	ok	1.25	0
EXT074+1	correct	VerifiedGood	ok	1.40	0
EXT075+1	incorrect	VerifiedBad	ok	1.15	0
EXT076+1	incorrect	VerifiedBad	ok	1.19	0
EXT077+1	correct	VerifiedGood	ok	1.29	0
EXT078+1	incorrect	VerifiedBad	ok	1.30	0
EXT079+1	incorrect	VerifiedBad	ok	1.03	0
EXT080+1	correct	VerifiedGood	ok	1.07	0
EXT081+1	incorrect	VerifiedBad	ok	1.10	0
EXT082+1	incorrect	VerifiedBad	ok	1.37	0
EXT083+1	correct	VerifiedGood	ok	1.45	0
EXT084+1	incorrect	VerifiedBad	ok	1.52	0
EXT085+1	incorrect	VerifiedBad	ok	1.18	0
EXT086+1	correct	VerifiedGood	ok	1.33	0
EXT087+1	incorrect	VerifiedBad	ok	1.53	0
EXT088+1	incorrect	VerifiedBad	ok	1.76	0
EXT089+1	correct	VerifiedGood	ok	1.66	0
EXT090+1	incorrect	VerifiedBad	ok	1.43	1
EXT091+1	correct	VerifiedGood	ok	1.51	0
EXT092+1	incorrect	VerifiedBad	ok	1.60	0
EXT093+1	correct	VerifiedGood	ok	1.32	0
EXT094+1	incorrect	VerifiedBad	ok	1.54	0
EXT095+1	incorrect	VerifiedBad	ok	1.50	0
EXT096+1	correct	VerifiedGood	ok	1.52	0
EXT097+1	edge	VerifiedBad	edge case	2.02	0
EXT098+1	edge	VerifiedBad	edge case	1.60	0
EXT099+1	correct	VerifiedGood	ok	1.38	0
EXT100+1	incorrect	VerifiedBad	ok	1.46	0
EXT101+1	correct	VerifiedGood	ok	1.16	0
EXT102+1	incorrect	VerifiedBad	ok	1.45	0
EXT103+1	edge	VerifiedBad	edge case	1.22	0
EXT104+1	edge	VerifiedGood	edge case	1.18	1
EXT105+1	edge	VerifiedBad	edge case	0.96	0
EXT106+1	correct	VerifiedGood	ok	0.94	0
EXT107+1	correct	VerifiedGood	ok	1.15	0
EXT108+1	correct	VerifiedGood	ok	1.28	0
EXT109+1	incorrect	VerifiedBad	ok	1.54	1

List of Figures

1 The ProoVer 2026 proof `PRV000+1.s` (comments and SZS delimiters omitted). Axiom and conjecture are imported from the problem file, `neg` negates the conjecture and one `consequence` step derives `$false`. 6
2 Architecture of TSTPy. Parsing, checking and verdict aggregation run in a worker process with a wall-clock limit. The internal rule checkers are grouped into families (Table 5). 11
3 Dependency graph of the valid benchmark proof `PRV089`. Each edge leads from a parent to the step derived from it, and the color shows which check validates the node. 12
4 Inference rules of the ProoVer set by the number of proofs that use them. Blue: internal checker. Red: decided by the prover. Purple: delegated, but some steps are decided by a sound shortcut. Rules used in only one proof are grouped by color. The labels give the number of steps. 20
5 Wall-clock time per proof of the ProoVer set against the number of steps. Proofs with a prover call are drawn in red, and the dashed line is the time limit. 21

List of Tables

1 Logical symbols of the FOF language (Sutcliffe 2026c). 5
2 Roles and statuses in the ProoVer 2026 proofs (Sutcliffe 2026c,d). 5
3 Systems entered in ProoVer 2026, based on their system descriptions (Cailler & Guilloud 2026a). 8
4 Checks of GDV, Nörgler and TSTPy, using the abbreviations of Taprogge et al. (2026b). "n.s.": not stated in the sources. 9
5 Rule families and the principle of their checks. 13
6 Verdicts, printed SZS status and score under the ProoVer 2026 scheme. 15
7 Groups of the TST set. 18
8 Verdicts on the 91 pairs of the ProoVer set and resulting score. The last row also includes the 9 deprecated proofs. 19
9 Size and statement coverage of the checker modules (measured with `pytest-cov`). 21
10 The six slowest proofs of the ProoVer set. "Interpreter" is the time from the start of the process until the verifier begins to check, that is, the start of the Python interpreter and the import of the modules. All phases come from the median repetition and add up to the total, and "MB" is the peak resident memory. 22
11 Mutation operators and outcomes. Every accepted mutant was checked with Vampire and is still valid. The detection rate is therefore the share of rejected mutants among those that are no longer valid. 23
12 Outcome for malformed inputs. `error` means that the parser rejected the file. Every input is processed in less than 0.2 s. 23
13 The ProoVer set with and without the external prover, under the three-valued and the binary verdict. No setting verifies an invalid proof, and the binary verdict has no *Unknown*. 23
14 Verifier on unmodified Vampire proofs. 26

15 Internal Skolem check and Vampire entailment check ($C \models P$) on correct, mutated and flawed Skolemization steps. The numbers are the steps that each check rejects. A perfect check rejects every mutated and flawed step and no control step. 27
16 Command-line options of TSTPy. 29
17 Results for each proof of the TST set (own test proofs). 30
18 Results for each proof of the Extra set (generated rule-level test proofs). 32

References

ANDREOTTI, BRUNO; HANNA LACHNITT; and HANIEL BARBOSA. 2023. Carcara: An efficient proof checker and elaborator for SMT proofs in the Alethe format. *TACAS 2023*, *LNCS*, vol. 13993, 367–386. Springer.

ASSAF, ALI; GUILLAUME BUREL; RAPHAËL CAUDERLIER; DAVID DELAHAYE; GILLES DOWEK; CATHERINE DUBOIS; FRÉDÉRIC GILBERT; PIERRE HALMAGRAND; OLIVIER HERMANT; and RONAN SAILLARD. 2023. Dedukti: a logical framework based on the $\lambda\pi$-calculus modulo theory. arXiv:2311.07185. URL `https://arxiv.org/abs/2311.07185`.

BARENDREGT, HENK, and FREEK WIEDIJK. 2005. The challenge of computer mathematics. *Philosophical Transactions of the Royal Society A* 363.2351–2375.

BODINGBAUER, JONAS; MÁRTON HAJDU; LAURA KOVÁCS; AXEL POLACZEK; and MICHAEL RAWSON. 2026. Lean on Vampire proofs (short paper). arXiv:2603.26342. URL `https://arxiv.org/abs/2603.26342`.

BÖHME, SASCHA, and TOBIAS NIPKOW. 2010. Sledgehammer: Judgement day. *Automated reasoning – IJCAR 2010*, *LNCS*, vol. 6173, 107–121. Springer.

CAILLER, JULIE, and SIMON GUILLOUD. 2026a. Proceedings of ProoVer 2026. `https://proover-competition.github.io/assets/ProoVer2026_proceedings.pdf`. Accessed: 2026-10-27.

CAILLER, JULIE, and SIMON GUILLOUD. 2026b. ProoVer 2026 results (detailed results, ProoVer section of the CASC-J13 result pages). `https://tptp.org/CASC/J13/WWWFiles/Results.html#PRVProblems`. Accessed: 2026-10-27.

CAILLER, JULIE, and SIMON GUILLOUD. 2026c. ProoVer: Website of the ProoVer competition. `https://proover-competition.github.io/`. Annual competition for proof checkers in the TPTP ecosystem. Accessed: 2026-10-09.

CHIHANI, ZAKARIA; DALE MILLER; and FABIEN RENAUD. 2013. Foundational proof certificates in first-order logic. *Automated deduction – CADE-24*, edited by Maria Paola Bonacina, *LNCS*, vol. 7898, 162–177. Springer.

CRUZ-FILIPE, LUÍS; MARIJN J. H. HEULE; WARREN A. HUNT JR.; MATT KAUFMANN; and PETER SCHNEIDER-KAMP. 2017. Efficient certified RAT verification. *Automated deduction – CADE-26*, *LNCS*, vol. 10395, 220–236. Springer.

DELAHAYE, DAVID; DAMIEN DOLIGEZ; FRÉDÉRIC GILBERT; PIERRE HALMAGRAND; and OLIVIER HERMANT. 2013. Zenon Modulo: When Achilles outruns the tortoise using deduction modulo. *Logic for programming, artificial intelligence, and reasoning – LPAR-19*, *LNCS*, vol. 8312, 274–290. Springer.

FITTING, MELVIN. 1996. First-order logic. *First-order logic and automated theorem proving*, Graduate Texts in Computer Science, 109–136. New York, NY: Springer New York. URL `https://doi.org/10.1007/978-1-4612-2360-3_5`.

GORDON, MICHAEL J.; ROBIN MILNER; and CHRISTOPHER P. WADSWORTH. 1979. *Edinburgh LCF: A mechanised logic of computation*, *LNCS*, vol. 78. Springer.

HAKEN, ARMIN. 1985. The intractability of resolution. *Theoretical Computer Science* 39.297–308.

HARRISON, JOHN. 2009. HOL Light: An overview. *Theorem proving in higher order logics (TPHOLs 2009)*, *LNCS*, vol. 5674, 60–66. Springer.

KOVÁCS, LAURA, and ANDREI VORONKOV. 2013. First-order theorem proving and Vampire. *CAV 2013*, *LNCS*, vol. 8044, 1–35. Springer.

NONNENGART, ANDREAS, and CHRISTOPH WEIDENBACH. 2001. Computing small clause normal forms. *Handbook of automated reasoning*, edited by John Alan Robinson and Andrei Voronkov, vol. 1, 335–367. Elsevier and MIT Press.

PLAISTED, DAVID A., and STEVEN GREENBAUM. 1986. A structure-preserving clause form translation. *Journal of Symbolic Computation* 2.293–304.

POLLACK, ROBERT. 1998. How to believe a machine-checked proof. *Twenty five years of constructive type theory*, edited by Giovanni Sambin and Jan M. Smith. Oxford University Press.

RIAZANOV, ALEXANDRE, and ANDREI VORONKOV. 2002. The design and implementation of VAMPIRE. *AI Communications* 15.91–110.

ROBINSON, JOHN ALAN. 1965. A machine-oriented logic based on the resolution principle. *Journal of the ACM* 12.23–41.

ROBINSON, JOHN ALAN, and ANDREI VORONKOV (eds.) 2001. *Handbook of automated reasoning*. Elsevier and MIT Press.

SCHULZ, STEPHAN; SIMON CRUANES; and PETAR VUKMIROVIĆ. 2019. Faster, higher, stronger: E 2.3. *CADE-27*, *LNCS*, vol. 11716, 495–507. Springer.

SCHURR, HANS-JÖRG; MATHIAS FLEURY; HANIEL BARBOSA; and PASCAL FONTAINE. 2021. Alethe: Towards a generic SMT proof format (extended abstract). *Proceedings of the 7th workshop on proof exchange for theorem proving (PxTP 2021)*, *EPTCS*, vol. 336, 49–54.

SUTCLIFFE, GEOFF. 2000. System description: SystemOnTPTP. *CADE-17*, *LNCS*, vol. 1831, 406–410. Springer.

SUTCLIFFE, GEOFF. 2006. Semantic derivation verification: Techniques and implementation. *International Journal on Artificial Intelligence Tools* 15.1053–1070.

SUTCLIFFE, GEOFF. 2007. TPTP, TSTP, CASC, etc. *Proceedings of the 2nd international computer science symposium in Russia*, edited by V. Diekert, M. Volkov, and A. Voronkov, *LNCS*, vol. 4649, 7–23. Springer.

SUTCLIFFE, GEOFF. 2008. The SZS ontologies for automated reasoning software. *Proceedings of the combined KEAPPA – IWIL workshops*, edited by Piotr Rudnicki, Geoff Sutcliffe, Boris Konev, Renate Schmidt, and Stephan Schulz, *CEUR Workshop Proceedings*, vol. 418, 38–49.

SUTCLIFFE, GEOFF. 2010. The TPTP world – infrastructure for automated reasoning. *Proceedings of the 16th international conference on logic for programming artificial intelligence and reasoning*, edited by E. Clarke and A. Voronkov, *LNAI*, vol. 6355, 1–12. Springer.

SUTCLIFFE, GEOFF. 2016. The CADE ATP system competition – CASC. *AI Magazine* 37.99–101.

SUTCLIFFE, GEOFF. 2017. The TPTP problem library and associated infrastructure: From CNF to TH0, TPTP v6.4.0. *Journal of Automated Reasoning* 59.483–502.

SUTCLIFFE, GEOFF. 2024. Stepping stones in the TPTP world. *Proceedings of the 12th international joint conference on automated reasoning*, edited by C. Benzmüller, M. Heule, and R. Schmidt, *LNAI*, vol. 14739, 30–50. Springer.

SUTCLIFFE, GEOFF. 2025. The 30th IJCAR automated theorem proving system competition (CASC-30). `https://tptp.org/CASC/30/`. Accessed: 2026-10-19.

SUTCLIFFE, GEOFF. 2026a. Semantic derivation verification (GDV). `https://tptp.org/Seminars/GDV/Contents.html`. Accessed: 2026-10-19.

SUTCLIFFE, GEOFF. 2026b. The SZS ontologies. `https://szs.tptp.org`. Accessed: 2026-10-19.

SUTCLIFFE, GEOFF. 2026c. The TPTP language: Syntax in BNF. `https://tptp.org/TPTP/SyntaxBNF.html`. Accessed: 2026-10-19.

SUTCLIFFE, GEOFF. 2026d. TPTP quick guide: The TPTP format for derivations. `https://tptp.org/UserDocs/QuickGuide/Derivations.html`. Accessed: 2026-10-19.

SUTCLIFFE, GEOFF. 2026e. The TSTP solution library. `https://tptp.org/TSTP/`. Accessed: 2026-10-19.

SUTCLIFFE, GEOFF; FRÉDÉRIC BLANQUI; and GUILLAUME BUREL. 2025. Proof verification with GDV and LambdaPi – it's a matter of trust. *Proceedings of the international FLAIRS conference*, vol. 38. URL `https://journals.flvc.org/FLAIRS/article/view/138642`.

SUTCLIFFE, GEOFF; STEPHAN SCHULZ; KOEN CLAESSEN; and ALLEN VAN GELDER. 2006. Using the TPTP language for writing derivations and finite interpretations. *IJCAR 2006*, *LNCS*, vol. 4130, 67–81. Springer.

TAPROGGE, MELANIE; FRÉDÉRIC BLANQUI; and ALEXANDER STEEN. 2026a. Formal verification of proofs from automated theorem provers for higher-order logic. URL `https://arxiv.org/abs/2609.24594`.

TAPROGGE, MELANIE; HAPPY SARIYANTO; and ALEXANDER STEEN. 2026b. A light-weight proof checker for TSTP refutations. *Practical aspects of automated reasoning*, *CEUR Workshop Proceedings*, vol. 4241. URL `https://ceur-ws.org/Vol-4241/`.

TSEITIN, G. S. 1983. On the complexity of derivation in propositional calculus. *Automation of reasoning 2: Classical papers on computational logic 1967–1970*, edited by Jörg H. Siekmann and Graham Wrightson, 466–483. Springer. Originally published in Russian in 1968.

WETZLER, NATHAN; MARIJN J. H. HEULE; and WARREN A. HUNT JR. 2014. DRAT-trim: Efficient checking and trimming using expressive clausal proofs. *Theory and applications of satisfiability testing – SAT 2014*, *LNCS*, vol. 8561, 422–429. Springer.